

SciML for AEC HS2021

Explainable AI methods for modelling pinch tests for reinforced concrete

Author(s)

Anrea Hauenstein

ETH-Nr.: 11-720-364

Parham Zendehtdel Nobari

ETH-Nr.: 18-951-111

Luca Angst

ETH-Nr.: 14-921-688

Supervisor:

Dr. Michael Anton Kraus , Dr. Danielle Griego

Date:

13.12.2021



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Explainable AI methods for modelling pinch tests for reinforced concrete

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):

Andrea

Nobari

Luca

First name(s):

Hauenstein

Parham Zendehdel

Angst

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date

Zurich, 13.12.2021

Signature(s)

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.

Contents

1	Motivation	1
2	Dataset	3
2.1	Dataset Description	3
2.2	Data Overview	3
2.3	Feature Engineering	4
3	Linear Regression Model	8
3.1	Simple Linear Regression	8
3.2	Non-Linear Regression.....	9
3.3	Linear Regressions with Penalty Functions	12
3.4	Summary of Linear/Non-Linear Regression Models	14
3.5	Comparing Eurocode to Non-Linear Regression Model	15
4	SVM Model.....	16
4.1	Linear kernel function	16
4.2	Polynomial kernel function.....	17
4.3	Radial basis kernel function.....	19
4.4	Comparison of the different kernel functions	21
4.5	Comparing Eurocode to RBF model.....	22
5	Decision Trees	23
5.1	One-Hot Encoding and Stratifying Data	23
5.2	Decision Tree Regressor Model	25
5.3	Random Forest Model	31
5.4	Comparing Eurocode to Random Forest Model.....	33
6	EuroCode	34
6.1	Sensitivity Analysis of Eurocode.....	35
7	Conclusion.....	37
8	Outlook	38
9	References	39
10	List of figures	40
11	List of tables.....	42

1 Motivation

While many aspects of structural behavior of reinforced concrete are based on fundamental theories and hence have a meaningful physical background, the calculation of punching resistance remains mainly empirical [1]. As a result, empirical factors and root terms appear in the formula to fit the experimental results. Such empirical formulas offer a big playground for machine learning models that try to improve the predictive power to better fit the experiments and maybe help to understand which parameters and interdependencies are at work. The goal of this project is exactly that: by exploring the field of scientific machine learning algorithms, more accurate models are constructed, tuned, analyzed, and compared.

If we calculate the punching resistance using the Eurocode formula and compare it to the measured resistance during experiments, we see that the calculated values fit the experimentally obtained results quite well when dividing by an apparent safety factor of $SF=1.8$. However, there is still a bit of a scatter for some datapoints on both the conservative as well as the unsafe side.

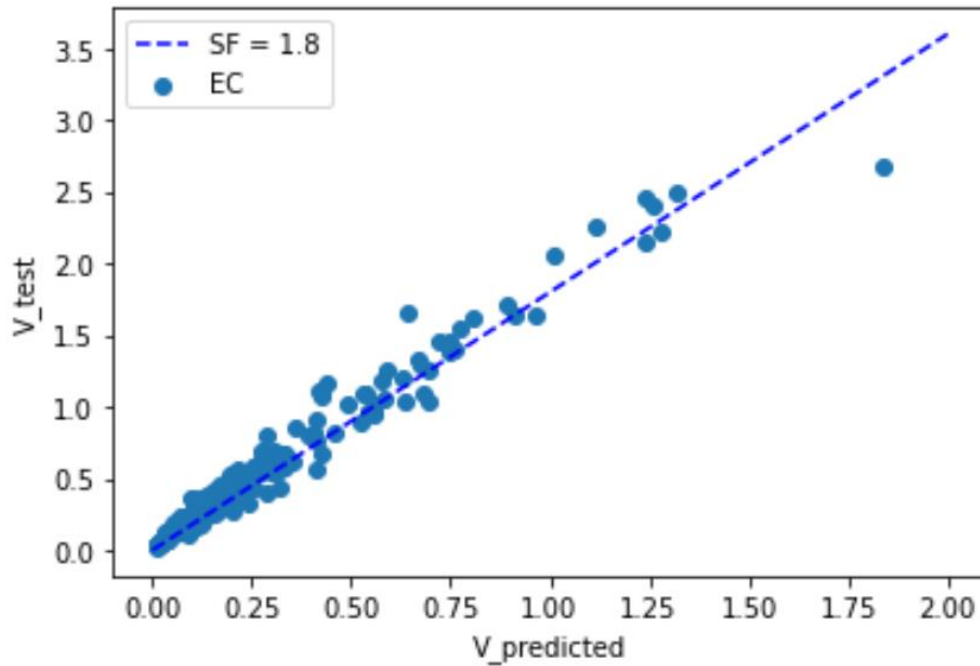


Figure 1: normal and adjusted EC formula

Our goal in delineating a new way to describe the punching behavior of slabs is to increase the accuracy in prediction and decrease the scatter as much as possible, so there would be no need for a big safety margin to compensate for unfavorable scatter and consequently more efficient structures could be built. In this project, a series of algorithm families are used to describe the punching behavior via regression and their accuracy is analyzed and compared.

Punching resistance is treated in the Eurocode as a series of differentiations. Depending on the slab type, column type and grid, load eccentricity etc., the formula varies. The used dataset however only has variations in column types. We are therefore not looking for a generalized formula including all possible variations but only the general formula with the allowed variation of the column type. As a result,

we do not expect a drastic increase in accuracy with the later proposed regression models. Nevertheless, it is interesting to explore alternative models to describe punching behavior. Especially linear regression models are not expected to increase the prediction accuracy simply because the equation offered on the Eurocode is of nonlinear nature. Still, a linear regression model would be a decent baseline model to compare all sorts of non-linear models to.

Non-linear models on the other hand offer a potential to increase the accuracy of punching behavior description. H. D. Nguyen et. al. proposed an XGBoost model with which they achieved to outperform predictions from the Eurocode 2 or other codes [2]. XGBoost seems to be a very promising algorithm that has already been proved to outperform other machine learning algorithms such as SVMs and are frequently used in the field of structural engineering because they are able to produce very accurate relationships between input and output. [2][3][4] Other linear and nonlinear regression algorithms have already been investigated in order to increase the accuracy of punching resistance prediction. S. Mangalathu et. al. found that tree-based algorithms outperform other algorithms by far and the performance difference between non-linear and linear non-tree-base algorithms is negligible. [4] A team of researchers even proposed alternative calculation formulas to increase the predictive power for punching strength [5]. The studied research papers did however not always include a balanced dataset and especially when it comes to the column types, they did not clearly prescribe their input parameters. In this report we aim at reconstructing and validating findings from above mentioned papers by constructing and tuning several regression models. While a lot of research in this field has already been done and is still ongoing, this project's goal is to get more familiar with machine learning algorithms and enhance our understanding about the model parameters, their strengths and weaknesses and their predictive power.

So, essentially, the questions we are addressing in this report are the following ones:

- What is the predictive power of the EC2 formula for punching resistance?
- Can ML models increase the accuracy of punching resistance prediction?

2 Dataset

2.1 Dataset Description

The aim of this project is to develop an algorithm to predict the punching resistance given the geometry and the material parameters. The dataset consists of 336 different test results executed from different researchers compiled by Dr. Karl Friedrich Siburg.

2.2 Data Overview

In Table 1 below one can see the different features assigned to each pinch test. Due to the fact that no failure mode is recorded, the interesting data are the numerical values. Furthermore, the amount of non-null values can give a first indication of how much data is actually available.

Table 1: Features of Dataset

0	Bez Vers.Ber	336 non null	object
1	Forscher	336 non null	object
2	Platte	336 non null	object
3	Last	336 non null	object
4	Lasteinleitung	336 non null	float64
5	h	336 non null	int64
6	d	336 non null	int64
7	Stütze	336 non null	object
8	c1	336 non null	int64
9	c2	14 non null	float64
10	rho_l	336 non null	float64
11	fcm,cyl	336 non null	float64
12	dg	204 non null	float64
13	fym	336 non null	int64
14	Esm	116 non null	float64
15	V_test	336 non null	float64

Lasteinleitung: Umfang der Lasteinleitungsfläche u_0 (/ Laststreifen b_s) [m]

h: Deckenstärke [mm]

d: Statische Höhe [mm]

c1: Seitenlänge / Durchmesser Stütze [mm]

c2: zweite Seitenlänge falls Stütze Rechteckig [mm]

rho_l: Längsbewehrungsgrad [%]

fcm,cyl: Zylinderdruckfestigkeit, Mittelwert [MPa]

dg: Grösstkorn Durchmesser [mm]

fym: Stahl Zugfestigkeit Mittelwert [MPa]

Esm: Stahl Steifigkeit Mittelwert [GPa]

V_test: Durchstanzlast [MN]

To get a fast overview of the missing data or any leakage of data, the heatmap in Figure 2 is used. The color yellow is an indication of missing data, and the color blue indicates available data.

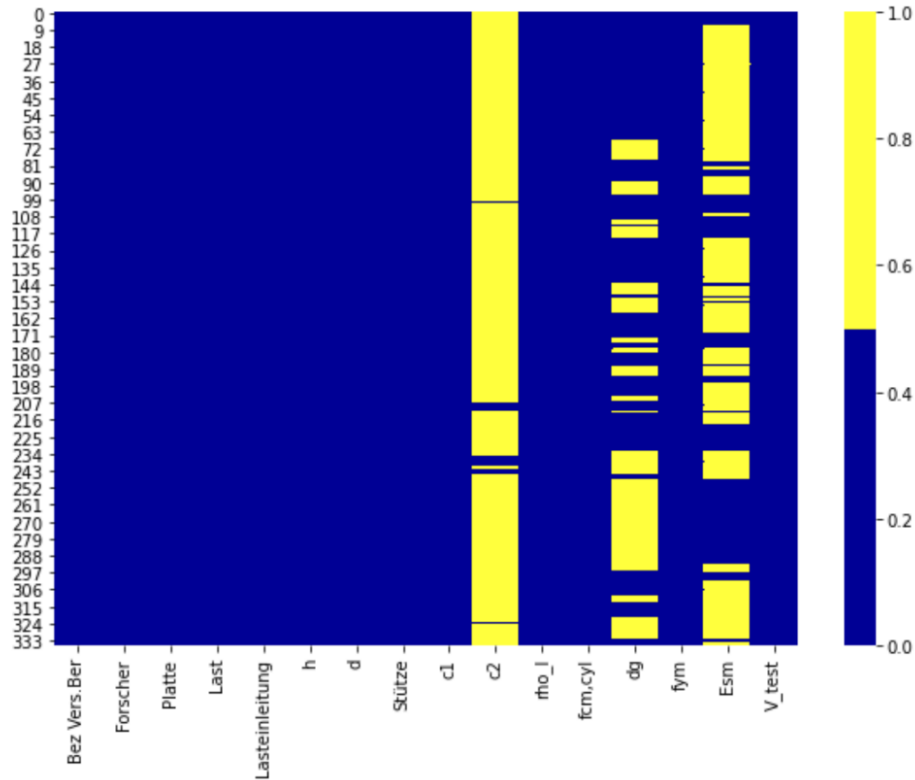


Figure 2: Heat-map showing available data (Yellow: missing data, Blue: available data)

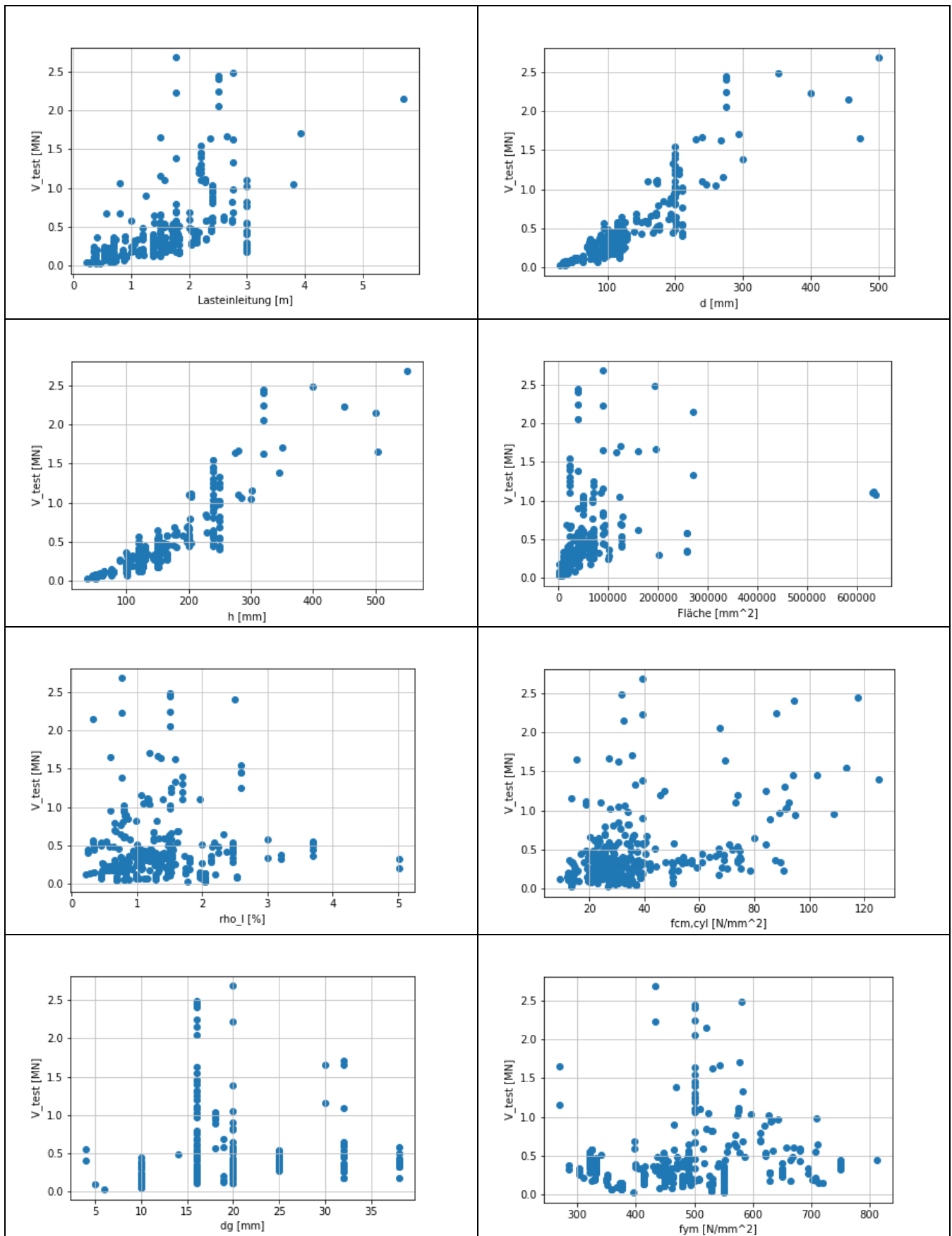
In a first step the non-numerical values are dropped out including *Bez*, *Vers.Ber*, *Forscher*, *Platte*, *Last* and *Stütze* (however the column *Stütze* is later used to generate a further feature). Since punching is a brittle failure and the E-Modulus of steel can be assumed rather similar over all reinforcements, the column *Esm* is dropped out.

2.3 Feature Engineering

Since the area of the column is important for the determination of the punching resistance and it is implicitly given by the column's *Stütze*, *c1* and *c2* features, this additional feature called *Fläche* is added to the dataset. This feature accounts for the area of the column.

The purpose of the plots below in Table 2 is to get a general intuition on how much the different features are correlated to the punching resistance. For the columns *h* and *d*, a certain correlation can be identified immediately, for $f_{cm,cyl}$ also a certain tendency can be explored. However, further investigations should be carried out to better explain the data.

Table 2: Data Overview



To better quantify the collinearity between two features the heat matrix Figure 3 is used. One can immediately see the high collinearity of h and d , since h has the higher variance, this feature is dropped out at this point. In addition, ρ_{h_l} , d_g and f_{ym} have a very poor collinearity with the searched punching resistance. The yielding strength of the reinforcement f_{ym} is dropped out due to the local brittle nature of failure, giving credence to the fact that the impact from yielding steel is not expected to be high. Due to the fact that, the scattering bandwidth of the maximal grain diameter is not very large in practice the feature is dropped out as well. Furthermore, a data leak of 39% is present on d_g and no tendency can be observed in a certain direction. The amount of reinforcement ρ_{h_l} is analyzed further by checking the multicollinearity to other features.

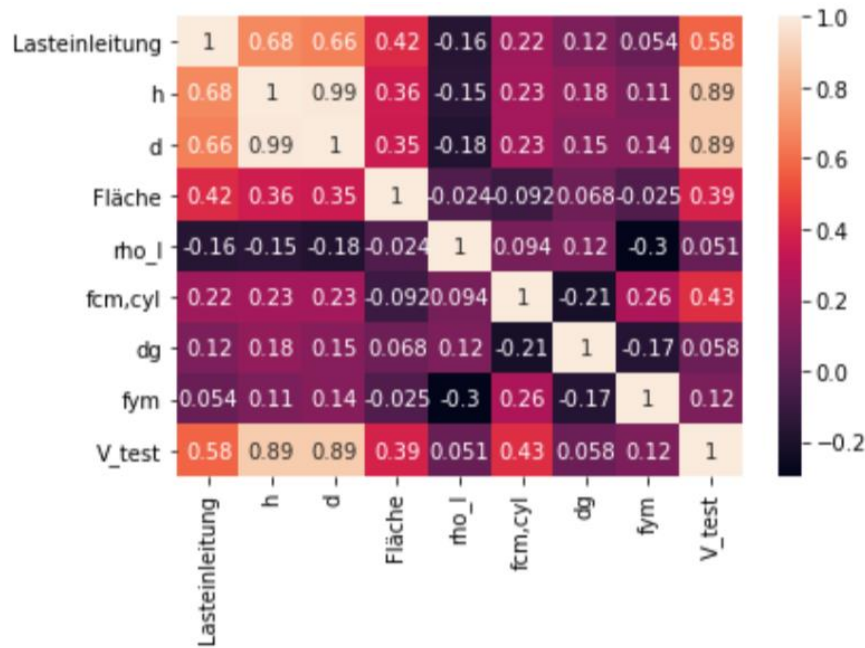


Figure 3: Colinearity Matrix

In the Table 3 below Variance Inflation Factor (VIF) is shown. This factor indicates multicollinearity between the features. As a general rule of thumb, a multicollinearity is present if VIF is higher than 5. The five features are assumed to have a certain relevance for developing an algorithm predicting shear resistance. However, for different algorithms different modifications can occur.

Table 3: Variance Inflation Factor

Lasteinleitung	2.057
d	1.944
Fläche	1.287
rho_l	1.107
fcmcyl	1.111

To get rid of size effects the dataset is processed with the MinMax scaler. The values of every feature are bounded in the range from 0 to 1. To detect any skewness of the data or outliers in the following the box-plots shown in Figure 5 as well as the histograms of the data (Figure 4) are mentioned. Generally, it can be observed, that most of the data is in the range from 0 to 0.4 in this range we also expect our algorithm to perform best.

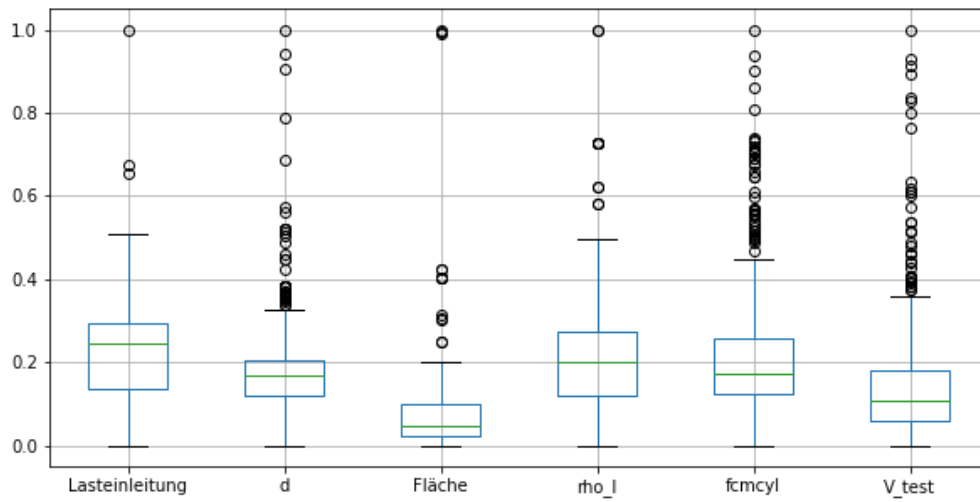


Figure 5: Box-plot of normalized features

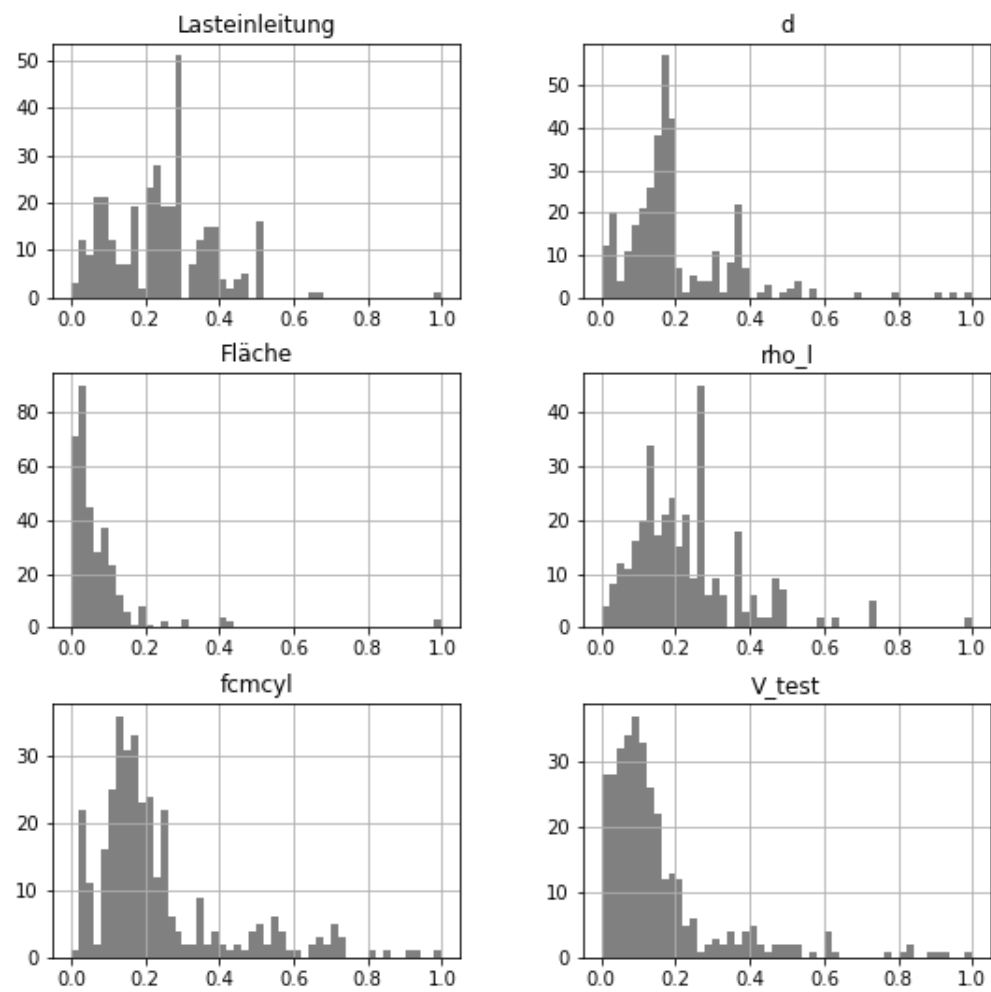


Figure 4: Histograms of normalized feature distribution

3 Linear Regression Model

As a first approach different types of linear regression models are investigated and the main results from each approach are shown in the following. The linear regression model is assumed to be a good point to start since it is an explainable algorithm, and the outcome can be interpreted more easily. During the process of building the algorithm it has been decided to drop out the feature *Lasteinleitung* since its contribution was very low.

3.1 Simple Linear Regression

As a general rule of thumb, the data is split into a 70%,30% ratio of training to testing subsets. Training subsets are used to train the regression model and test subsets are used to evaluate the predictive performance of the model.

For a linear regression model, the following formula is optimized:

$$V_{\text{test}} = \beta_1 * d + \beta_2 * \text{Fläche} + \beta_3 * \text{rho_l} + \beta_4 * f_{\text{cm,cyl}}$$

Figure 6 below shows an overview of the results. As relevant values the R-squared value of 0.899 can be seen as quite good first fit. Further the coefficients of the features as well as the intercept coefficient show that feature *d* has the highest coefficient, meaning the highest importance on the output. The column $P > |t|$ shows how big the probability is that the feature is wrongly going to have an impact on the V_{test} .

OLS Regression Results						
=====						
Dep. Variable:	V_test	R-squared:	0.899			
Model:	OLS	Adj. R-squared:	0.897			
Method:	Least Squares	F-statistic:	511.5			
Date:	Tue, 30 Nov 2021	Prob (F-statistic):	3.46e-113			
Time:	10:34:51	Log-Likelihood:	368.67			
No. Observations:	235	AIC:	-727.3			
Df Residuals:	230	BIC:	-710.0			
Df Model:	4					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	-0.1318	0.009	-14.713	0.000	-0.149	-0.114
d	0.9251	0.026	35.079	0.000	0.873	0.977
Fläche	0.2141	0.029	7.451	0.000	0.157	0.271
rho_l	0.1834	0.024	7.657	0.000	0.136	0.231
fcmcyl	0.2082	0.020	10.493	0.000	0.169	0.247
=====						
Omnibus:	87.415	Durbin-Watson:	2.100			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	611.872			
Skew:	1.266	Prob(JB):	1.36e-133			
Kurtosis:	10.488	Cond. No.	11.0			
=====						

Figure 6: Summary of linear regression model

Applying the model to the test data and compare the values with the one from the dataset we observe Figure 7. Further a slight nonlinearity can be observed from 0.4 onwards therefore a nonlinear model is investigated next.

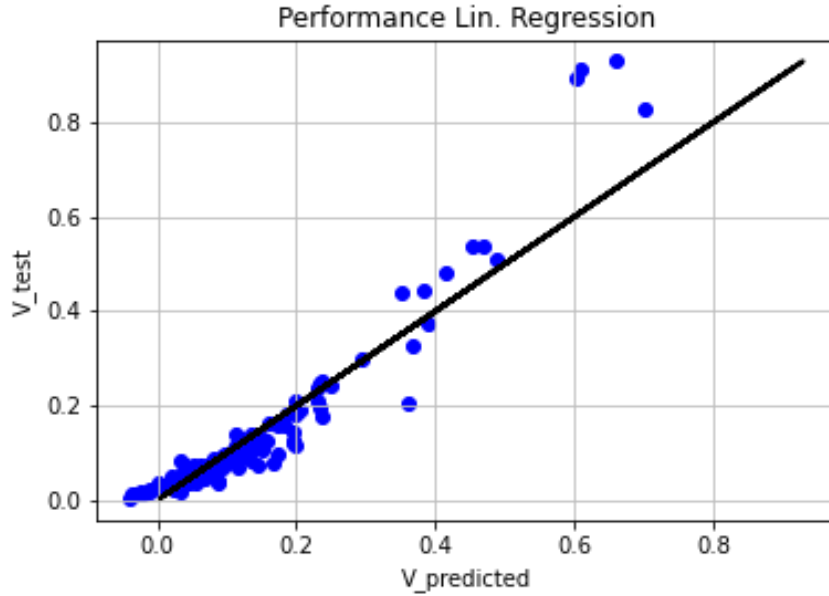


Figure 7: Prediction of linear regression model (MSE=0.004092)

3.2 Non-Linear Regression

Here the same procedure as in the linear case is repeated but by multiplying different features and by doing so iteratively, the most relevant multiplication is found and the resulting function to optimize is the following:

$$V_{test} = \beta_1 * d + \beta_2 * Fläche + \beta_3 * rho_l + \beta_4 * d * f_{cm,cyl}$$

The interpretation scheme is the same, one can observe an improved R-squared value of 0.926. The linear term containing $f_{cm,cyl}$ is not taken into account since the coefficient is very small (see Figure 8).

```

=====
                        OLS Regression Results
=====
Dep. Variable:          V_test      R-squared:                0.926
Model:                  OLS         Adj. R-squared:           0.925
Method:                 Least Squares   F-statistic:              574.7
Date:                  Tue, 30 Nov 2021   Prob (F-statistic):       2.11e-127
Time:                  10:34:52         Log-Likelihood:           405.56
No. Observations:      235            AIC:                     -799.1
Df Residuals:          229            BIC:                     -778.4
Df Model:               5
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.0719	0.010	-7.140	0.000	-0.092	-0.052
d	0.6735	0.035	18.978	0.000	0.604	0.743
Fläche	0.2374	0.025	9.594	0.000	0.189	0.286
rho_l	0.1698	0.021	8.252	0.000	0.129	0.210
fcmcyl	-0.0748	0.035	-2.127	0.034	-0.144	-0.006
fcmcyl:d	1.1313	0.123	9.191	0.000	0.889	1.374

```

=====
Omnibus:                47.011      Durbin-Watson:           2.083
Prob(Omnibus):          0.000      Jarque-Bera (JB):        240.405
Skew:                   0.639      Prob(JB):                6.26e-53
Kurtosis:               7.788      Cond. No.                49.1
=====

```

Figure 8: Summary of non-linear regression model

Using this non-linear model, the values in the scaled range of 0.3 till 0.6 can be captured more accurately. Generally, the values are predicted more accurately with an MSE of 0.002473, which is approximately a reduction of 40 % with respect to the simple linear regression. For the values above 0.8 (scaled value) the model prediction is on the safe side, because the values are lying above the 45° line in Figure 9. In Figure 10 a measure of how important the different features are is stated, as the impact on the model increases its importance is bigger. Due to the non-linearity one can observe, that the impact of the concrete property $f_{cm,cyl}$ might be slightly increased compared to the simple linear regression. The statical depth d is still the dominating feature of the model.

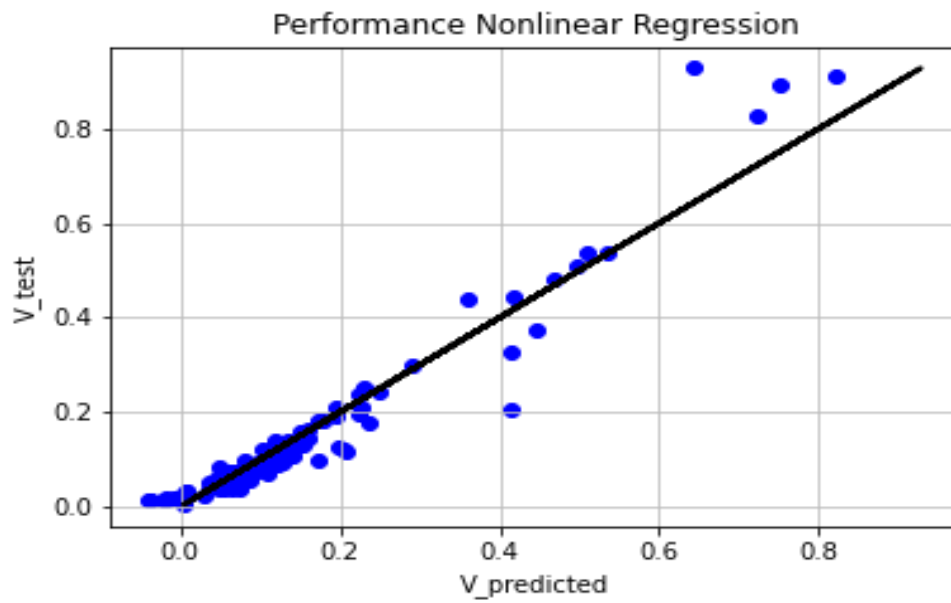


Figure 9: Prediction of non-linear regression model (MSE=0.002473)

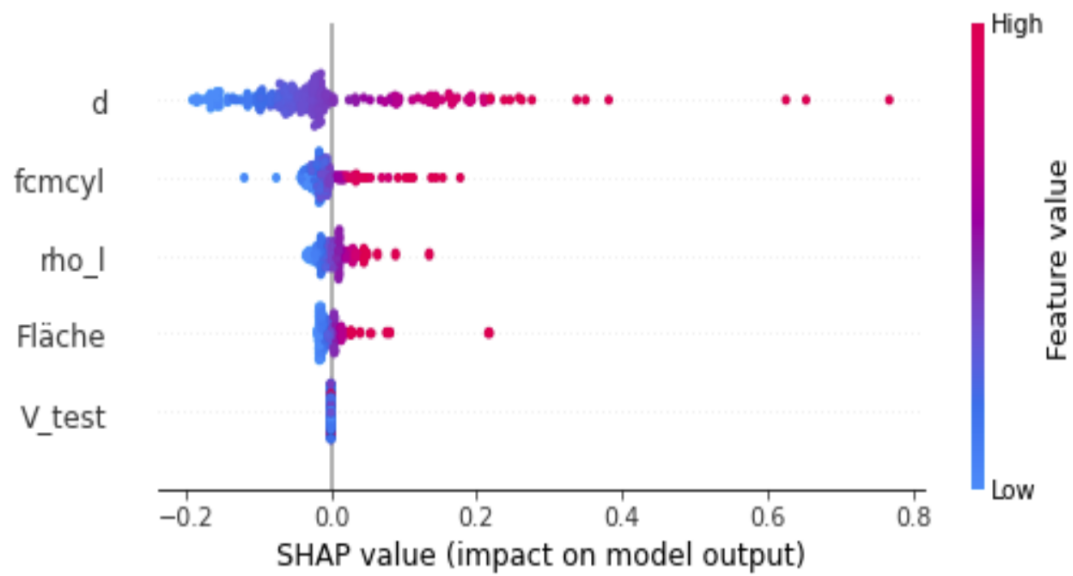


Figure 10: Shapely Values of non-linear regression model

3.3 Linear Regressions with Penalty Functions

In the following Lasso, Ridge and Elastic Net Regressions Models are investigated. At this point, not all models will be discussed in detail, but a summary of the results and the findings will be presented. Since the Lasso model is a sparse model, some features coefficient are driven to zero if they are of minor relevance. It is observed, that the feature coefficient `Lasteinleitung` decays to zero. The hyperparameters of the three models are tuned with a grid search algorithm, this tuning was done for different random states. Based on the different results observed a mean value for the hyperparameter was than chosen. As a result, the predicted values of the three models are plotted against the values from the test split in the following figures. Generally, the same non-linearity can be observed as in the simple linear regression mode, the MSEs are all of a similar range while the Lasso model has the lowest MSE of the three.

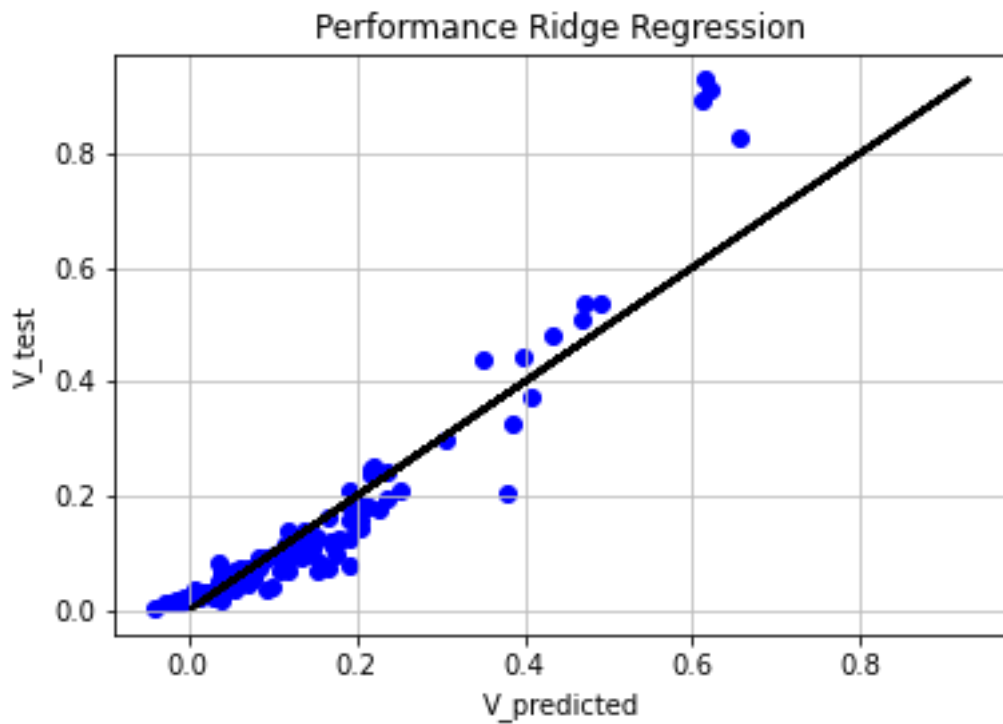


Figure 11: Prediction of ridge regression model (MSE=0.004579)

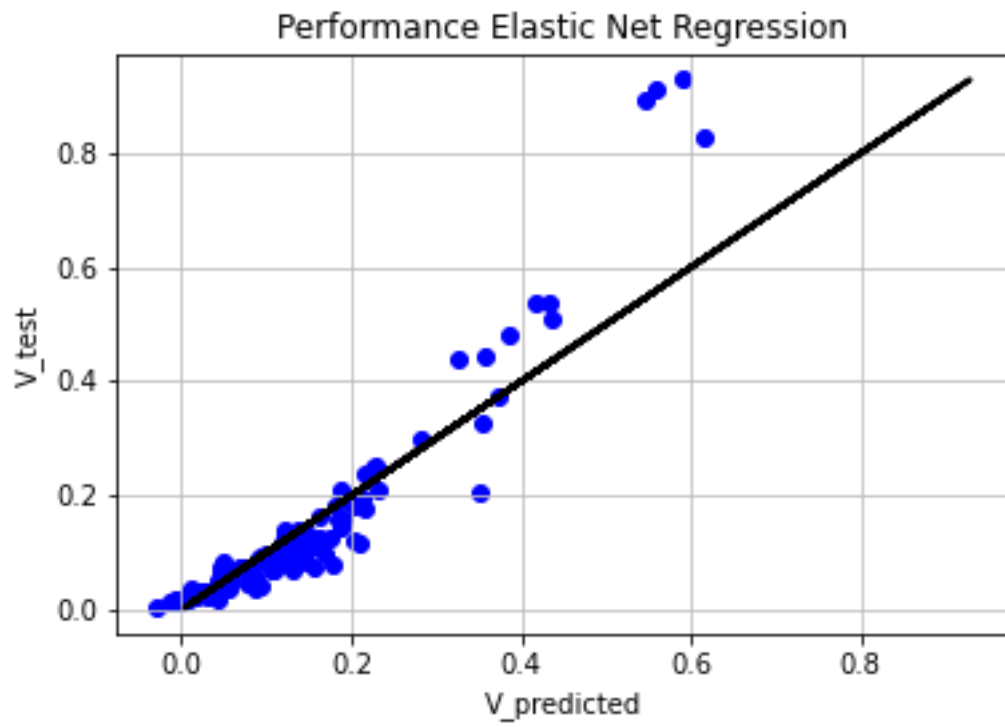


Figure 12: Prediction of elastic net regression model (MSE=0.005737)

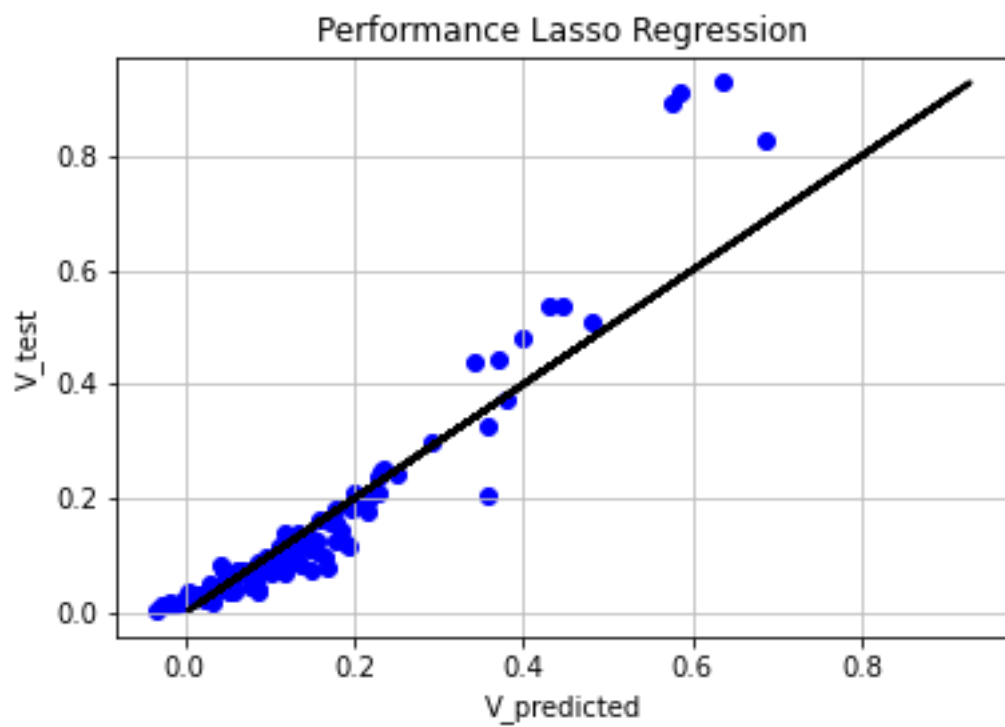


Figure 13: Prediction of lasso regression model (MSE=0.004540)

3.4 Summary of Linear/Non-Linear Regression Models

Comparing the errors of the different models based on training and test data, we see that the non-linear regression model performs the best.

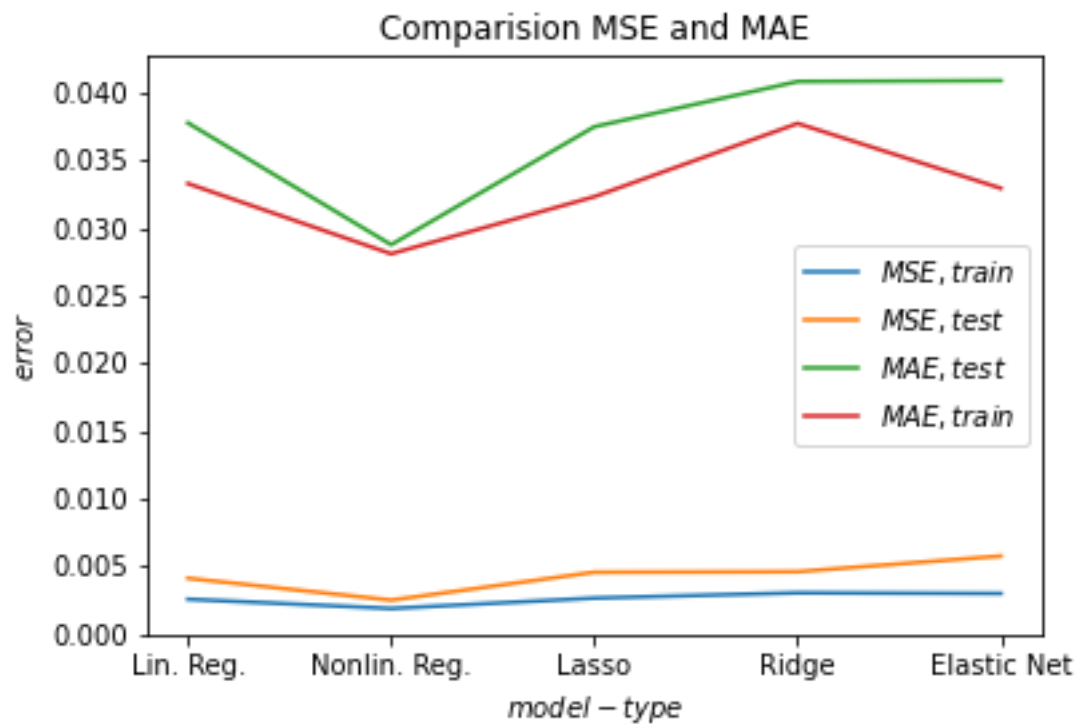


Figure 14: Comparison of different linear and non-linear regression models

3.5 Comparing Eurocode to Non-Linear Regression Model

To compare the two models the safety factor of 1.5 is removed from the Eurocode and $f_{ck} = f_{cm,cyl} - 8$ [N/mm^2] is used. On this basis the two models are compared. The blue points show the prediction of the model against the Eurocode values. If we fit the values to the $V_{prediction} = V_{Rd_Code}$ axis we observe a factor of 1.23 with a small negligible intersection offset. This fitted values are shown in green. In conclusion we can say that using the developed non-linear regression model we can increase the punching resistance by 20 % with respect to the Eurocode.

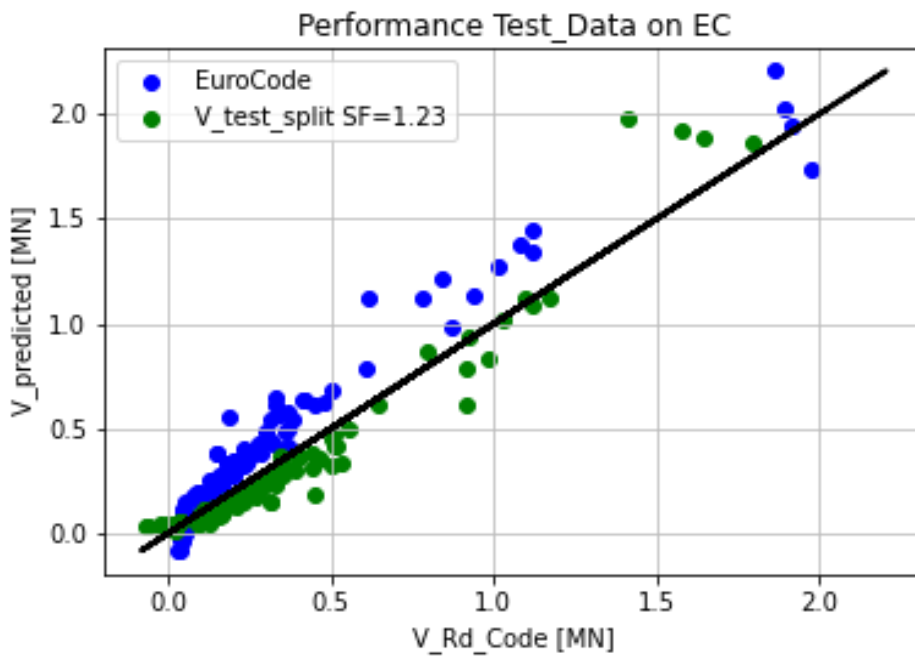


Figure 15: Non-linear regression model predictions vs. Eurocode values

4 SVM Model

Using the sklearn library, a support vector machine model was established to solve the regression problem. In order to get balanced train and test datasets, the sets were created by stratifying with the y-Value (V_{test}). By doing so, it is guaranteed that the model is trained on V_{test} values over the whole range from low to high values and hence will later be able to predict values over the whole range. According to the general rule, the dataset is split into 70% training and 30% test data.

The hyperparameters were tuned in an iterative way by first using GridsearchCV to get a first estimate of the parameters. Then, in a time expensive parameter study, the hyperparameters were adjusted to get an optimal minimization of both the MSE and the number of support vectors. Moreover, different kernel functions were analyzed.

4.1 Linear kernel function

The linear kernel was chosen to compare it to the linear regression models. Since the Eurocode formula is highly nonlinear and describes the experimental values quite accurately, the performance of the linear kernel model is not expected to show an increased accuracy. Nevertheless, it is a good model to study and compare to simple linear regression. When validating the tuned model against the unseen test data, the model performance is illustrated in **Error! Reference source not found.**. The model reached an accuracy measured as MSE of 0.0029 which is slightly better than what the linear regression models achieved (see section 3).

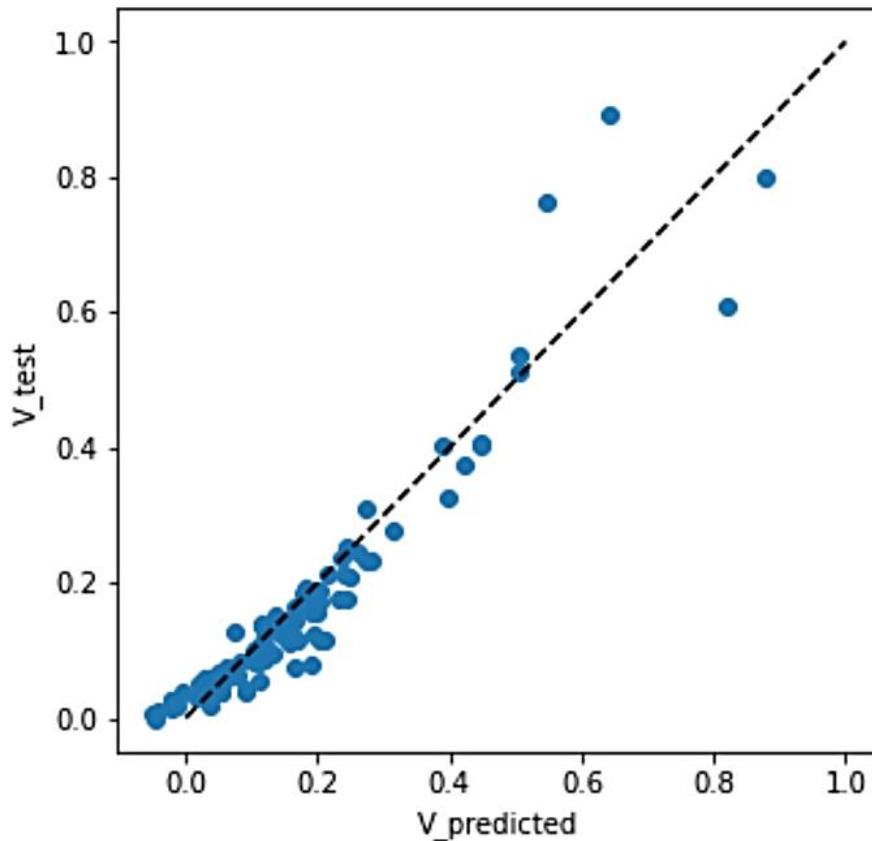


Figure 16: Model performance for the linear kernel function

In the following, the support vectors are illustrated and compared to the rest of the data in Figure 17. Because the input vector is four-dimensional, two two-dimensional cuts are illustrated to represent the hyperspace. The green scatter plots show the input data and the red crosses represent the data points that were chosen by the model as support vectors to create the hyperplane used for the regression. As the plots show, the support vectors are capturing parameter values in the whole range for all input variables. This is an indication of a robust model that is not overfitted towards certain parameter values.

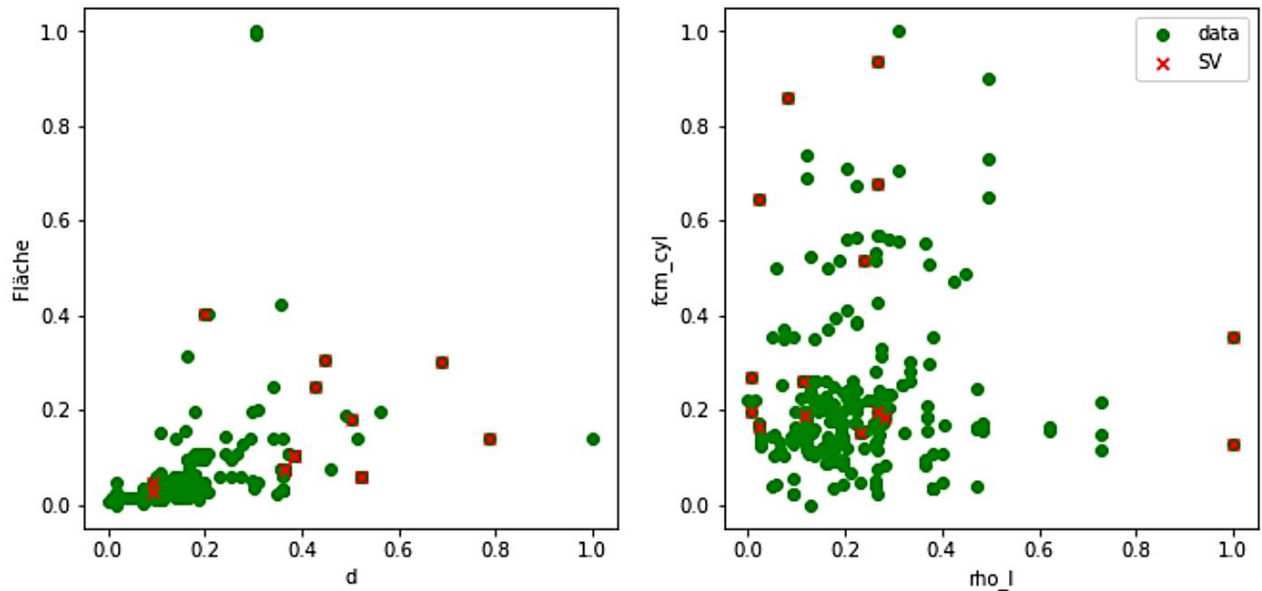


Figure 17: Scatterplot of input variables and support vectors for the linear kernel model

4.2 Polynomial kernel function

The polynomial kernel was chosen with a third-degree polynomial function. With this polynomial kernel function, the inherent nonlinear nature of the punching resistance values may be captured. The polynomial degree was restricted to the third order to counteract overfitting and remain an interpretable polynomial function. When validating the tuned model against the unseen test data, the model performance is illustrated in Figure 19. The model reached an accuracy measured as MSE of 0.0024 which is only slightly more accurate than the model with linear kernel function. The chosen support vectors to construct the hyperplane are plotted in Figure 18 with red crosses, while the green scatters represent the input data. As the plots show, the support vectors are capturing parameter values in the whole range for all input variables. This is an indication of a robust model that is not overfitted towards certain parameter values.

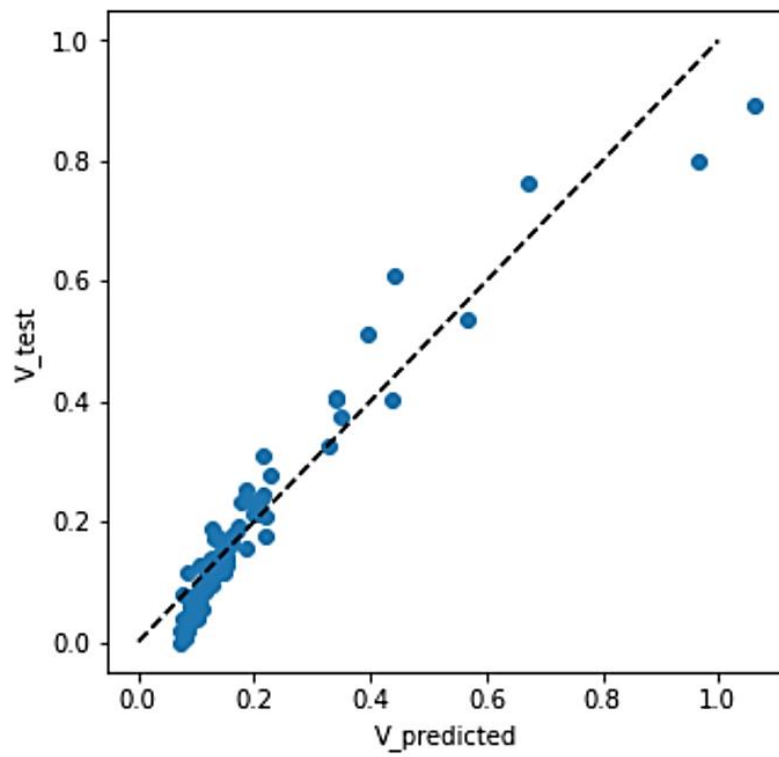


Figure 19: Model performance for the polynomial kernel function

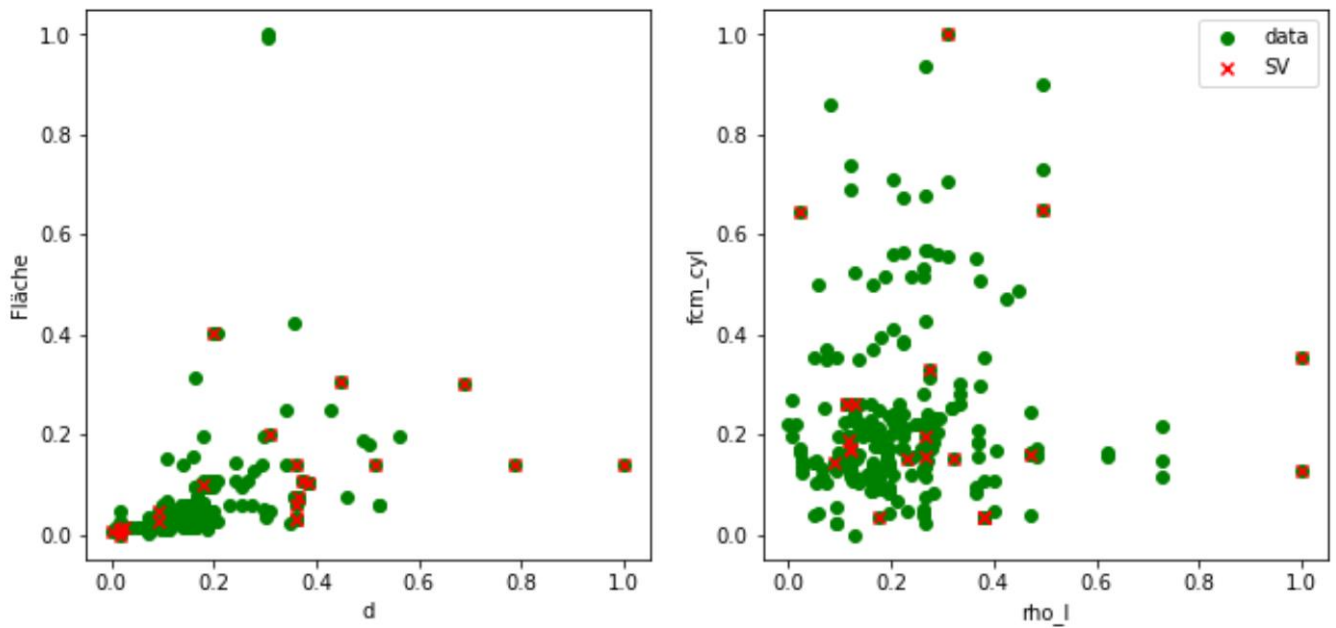


Figure 18: Scatterplots of input variables and support vectors for the polynomial kernel model

4.3 Radial basis kernel function

The radial basis function (RBF) was chosen because this kernel promises good performance for high nonlinearity. For this kernel function, the parameter tuning was especially time consuming because the number of support vectors can not be limited (or at least we did not manage to implement this). Consequently the hypertuning was made manually by incrementally adjusting the three parameters epsilon, gamma and C and optimizing these parameters in order to minimize both the MSE and the number of used support vectors. When validating the tuned model against the unseen test data, the model performance is illustrated in Figure 20. The model reached an accuracy measured as MSE of 0.0024 which is only slightly more accurate than the model with linear kernel function. The chosen support vectors to construct the hyperplane are plotted in Figure 1 with red crosses, while the green scatters represent the input data. As the plots show, the support vectors are capturing parameter values in the whole range for all input variables. This is an indication of a robust model that is not overfitted towards certain parameter values.

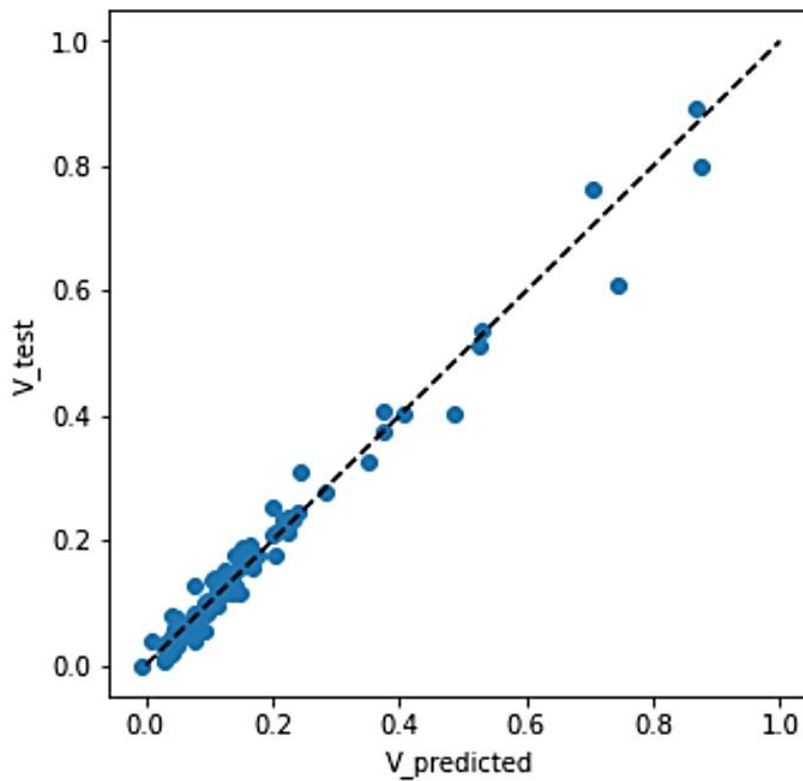


Figure 20: Model performance for the radial basis kernel function

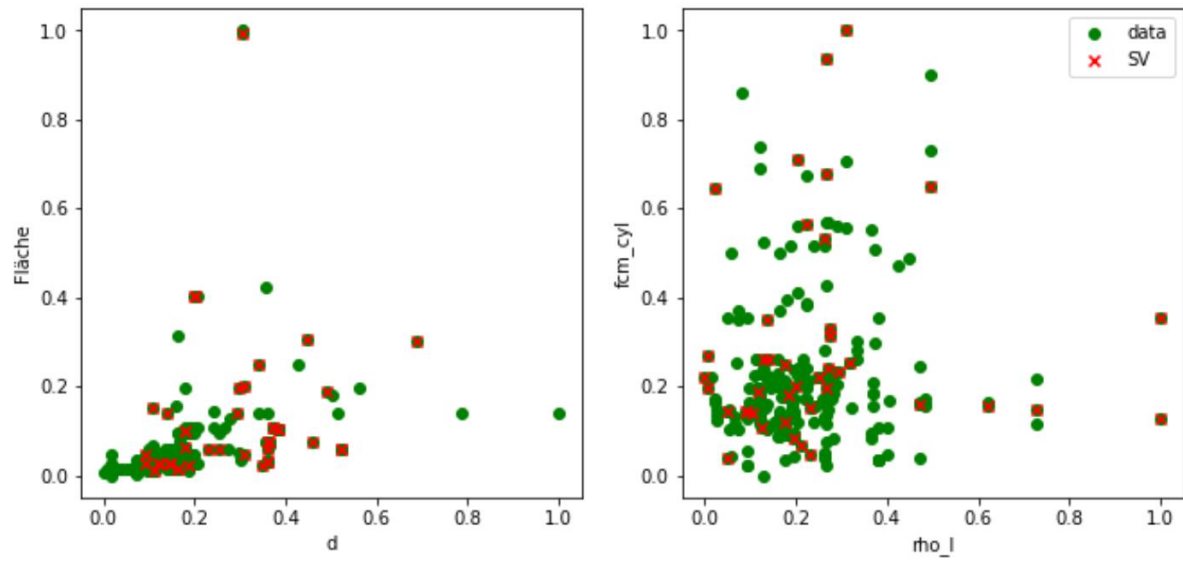


Figure 21: Scatterplots of input data and support vectors for the RBF kernel model

4.4 Comparison of the different kernel functions

In this section the three different kernel function models are analyzed and compared. The parameters listed in Table 4 were obtained by minimizing the amount of SV and MSE.

A high C value (regularization parameter) indicates an overfitting because outliers are punished heavily. A low C value creates smoother boundaries but cannot classify outliers correctly. This is represented by the low C value in the linear kernel model and the resulting little amount of support vectors needed to construct the hyperplane.

The gamma value controls the degree of nonlinearity of the kernel function and represents almost linear models with low values. High values indicate an overfitting because the support vectors mainly represent themselves and not the rest of the data.

Epsilon defines a margin of tolerance for which no penalties are given in the cost function. A higher value increases the generalization and hence less SVs are needed, but outliers will be classified wrong. This trend can be seen when comparing the polynomial with the RBF kernel model.

A high amount of support vectors indicates an overfitting because the data is simply learned by heart and no patterns are identified. Hence, a lot of support vectors are needed to create a hyperplane. In maximal overfitting, every data point is represented by an individual support vector, and no regression fit was made at all.

Table 4: Optimized model hyperparameter and their effect on the model accuracy

Kernel	C	gamma	epsilon	SV	MSE
Linear	0.7	-	-	15	0.0029
Polynomial	18	1.12	0.07	23	0.0024
RBF	6.15	1.2	0.03	41	0.0007

With 41 support vectors the RBF model uses ca. 18% of the input vectors as support vectors. The linear kernel SVR shows similar MSE as seen with linear regression models discussed above, which makes perfect sense. As the C-value in the polynomial kernel SVR is so high, the model is prone to overfitting the test data, which is represented in the only slightly improved MSE performance compared to the linear kernel model. The RBF proves to be the best performing model, since the MSE is a factor of 3 smaller than the other kernel models. However, the RBF model shows the least regularization because the highest number of SVs was chosen.

The RBF (radial basis function) uses a nonlinear kernel which is well-suited for cluster identification. As the comparison showed, this kernel performs best.

4.5 Comparing Eurocode to RBF model

To compare the two models the safety factor of 1.5 is removed from the Eurocode and $f_{ck} = f_{cm,cyl} - 8 \text{ [N/mm}^2\text{]}$ is used. On this basis the two models are compared. The blue points show the prediction of the model against the Eurocode values. If we fit the values to the $V_{\text{prediction}} = V_{\text{Rd_Code}}$ axis we observe a factor of 1.27 (See Figure 22). This fitted values are shown in green. In conclusion we can say that using the developed RBF model we can increase the punching resistance by almost 30 % with respect to the Eurocode.

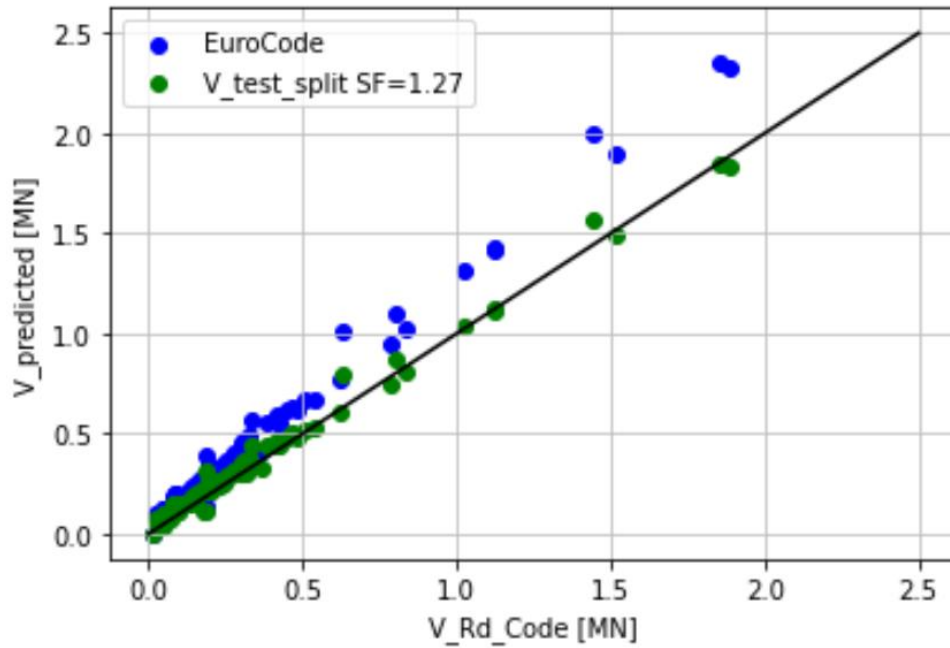


Figure 22: RBF model predictions vs. Eurocode values

5 Decision Trees

5.1 One-Hot Encoding and Stratifying Data

The third type of models which are used to predict the shear punching resistance are the decision tree models. Decision trees are easily scalable and interpretable and give insight into feature importance, this is why we investigate whether the cross-section profile shape of the test elements which is represented by the categorical variable `Stütze`. In order to do so, the categorical variable `Stütze` is one-hot encoded into three separate features, namely: `Stütze_k` (representing circular profiles), `Stütze_q` (representing square profiles) and `Stütze_r` (representing rectangular profiles).

For the purposes of our project, we opted for decision tree regressors since the variable we're trying to predict is a continuous variable. As a further step, the data is split into test and train subsets. To avoid imbalanced subsets, the splitting of the dataset is accompanied by the `stratify` command along the `Forscher` variable. This ensures that the subsets have an equal distribution of input from researchers and that the model is not biased towards any particular researcher. However, the stratification won't work if there researchers with only one experiment are present in the dataset. This is why researchers with less than 1 data point were excluded from the dataset. Figure 23 shows the distribution of experiments among all researchers and Figure 24 shows the distribution after the exclusion of researchers with less than 2 experiments.

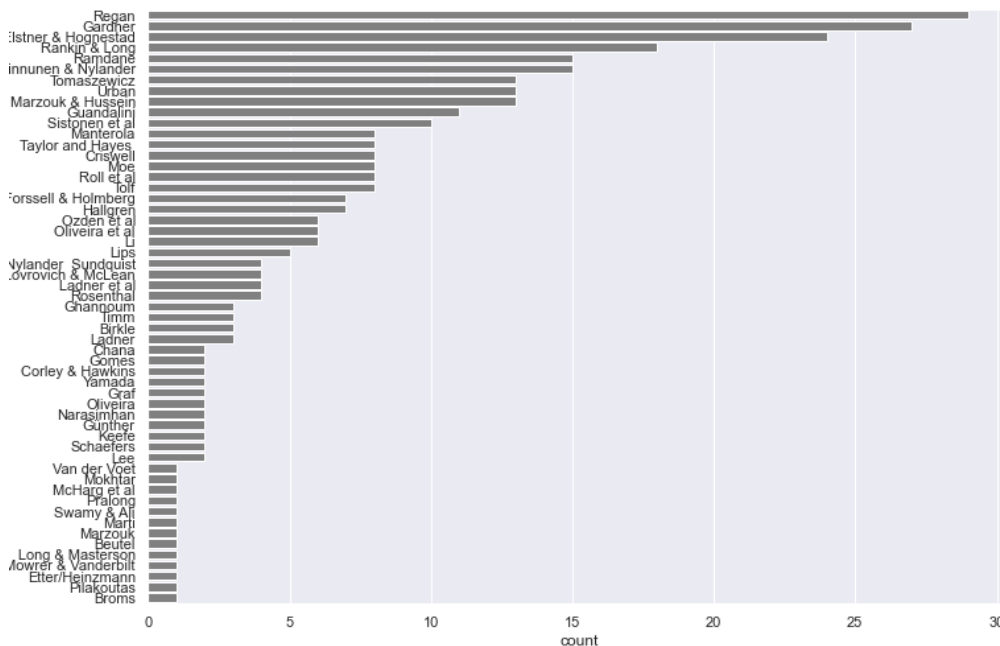


Figure 23: Distribution of experiment counts among researchers in dataset

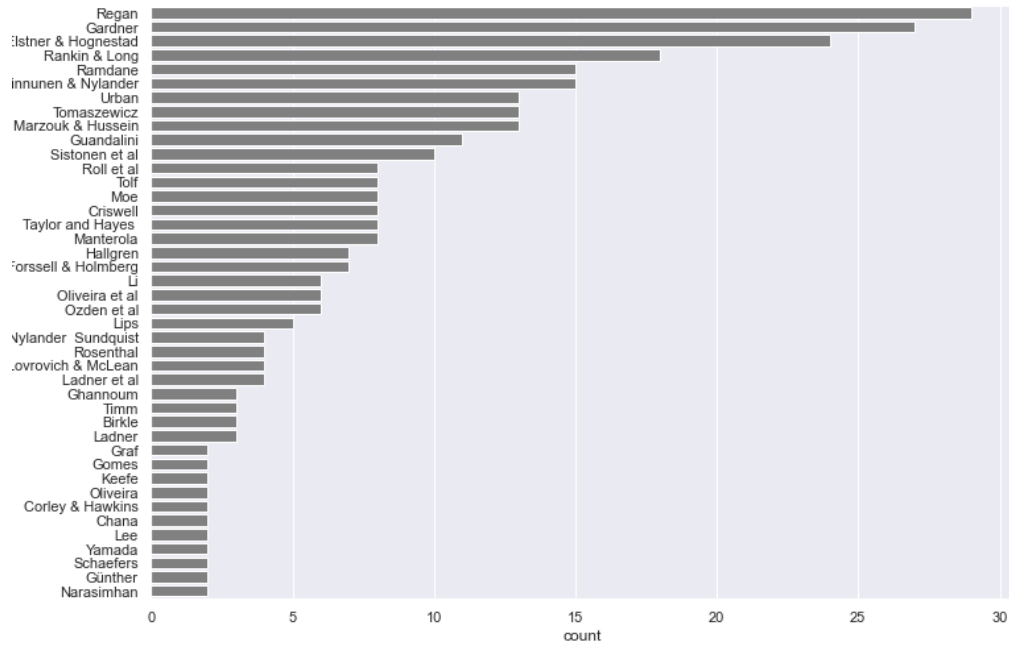


Figure 24: Distribution of experiment counts among filtered researchers in dataset

After splitting the dataset into train and test subsets and by comparing Figure 24 to Figures 25 and 26 it is observed that the distribution of researchers in the train and test subset is similar to that of the entire dataset.

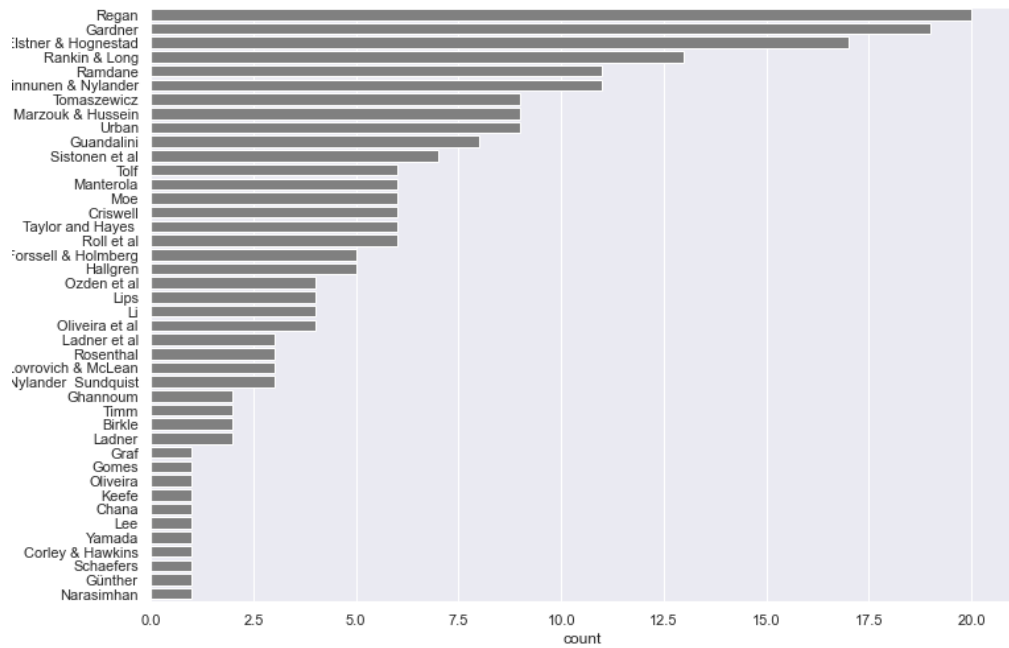


Figure 25: Distribution of experiment counts among filtered researchers in training subset

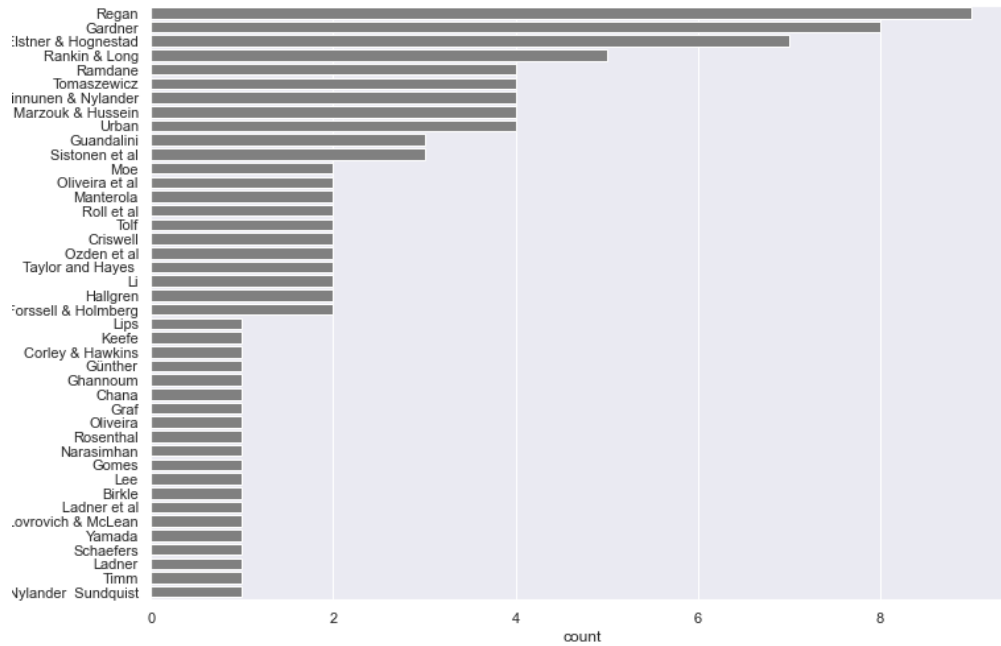


Figure 26: Distribution of experiment counts among filtered researchers in testing subset

5.2 Decision Tree Regressor Model

As a first step, a small and simple decision tree regressor model is created in order to make predictions. Even though the initial tree model is too small to cover all the data variance, one can observe that the simple decision tree model starts splitting the data based on the feature `d` and `Fläche` further down the line. This is an indication that feature `d` is probably going to be the most consequential feature moving forward. The initial decision tree is plotted in Figure 27 and gives information about the splitting sequence, number of samples which the model based its splits on and the MSE of each node.

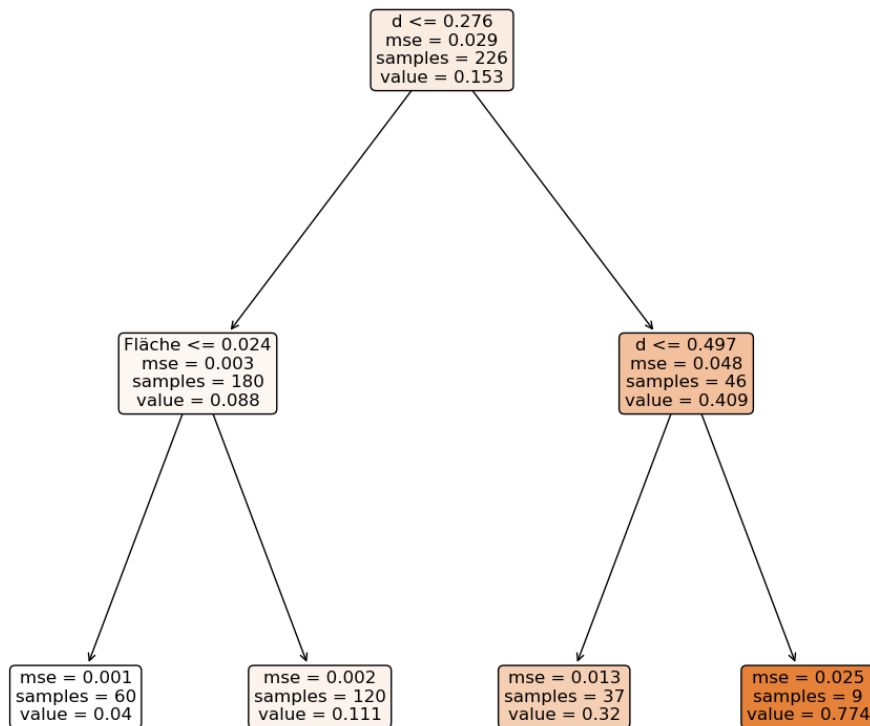


Figure 27: Simple decision tree with 3 levels and 7 nodes, training MSE=0.00437, testing MSE=0.00743

As a second step, we create a large tree in order to capture all the variance in the training dataset.

This results in much better training and testing accuracy, with an improvement of 61.55% in training MSE and 50.07% in testing MSE. The large tree model is shown in Figure 28 and the predictive performance of the large tree is shown in Figure 29.

Even though this model seems great, there is a risk of model overfitting which is addressed in the next step. As a final step, the tree regressor model is tuned by changing the hyperparameters `ccp_alpha` (for pruning the decision tree), `min_samples_leaf` (minimum samples per leaf), `min_samples_split` (minimum samples required for splitting) and `max_depth` (the maximum depth of the decision tree). The hyperparameter tuning is achieved using a 5-fold cross validation grid search which ensures that the resulting model does not overfit. 57600 model fits are carried out in this exhaustive search for parameter values for the best estimator. Table 5 shows all the hyperparameters and predictive performance of the three different decision trees.

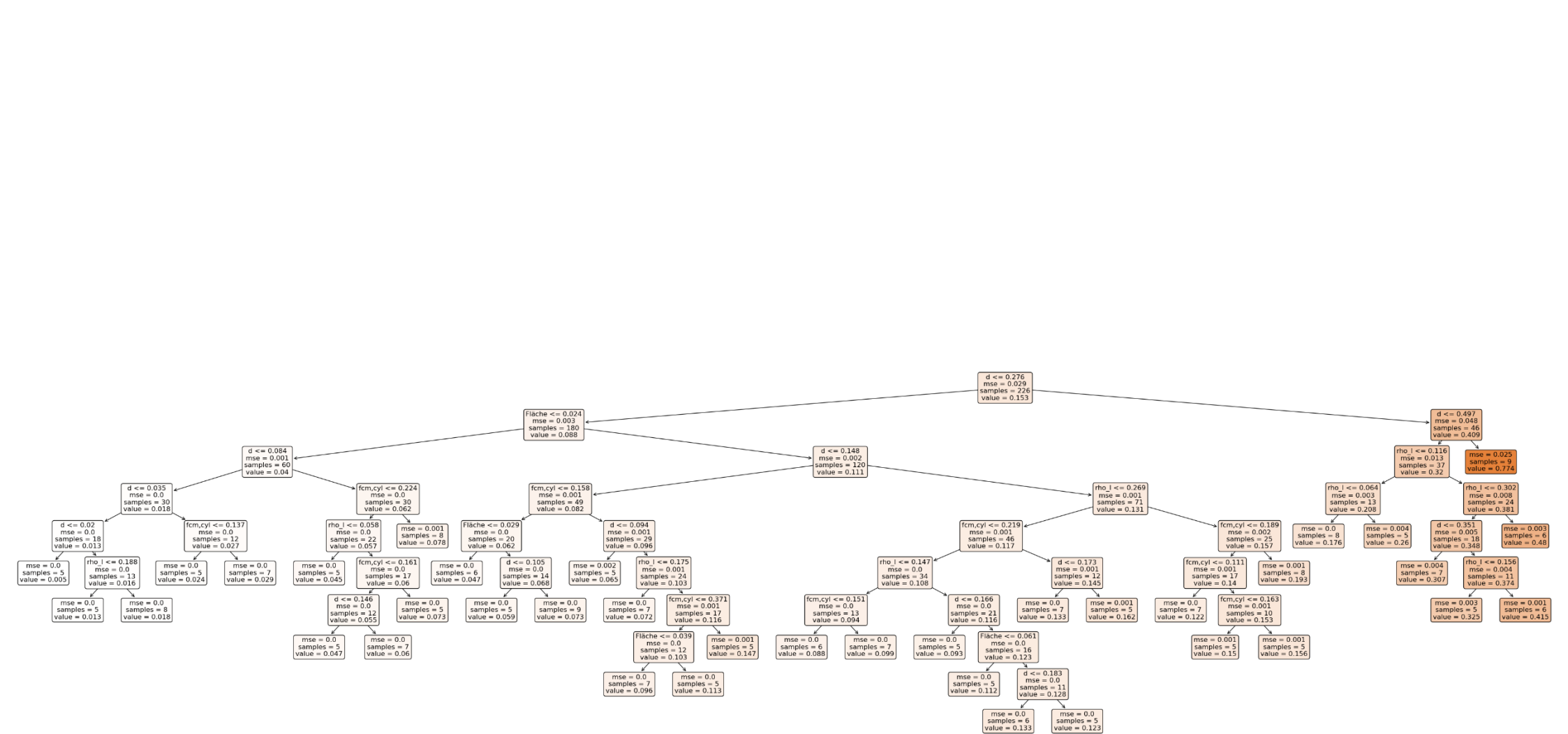


Figure 28: Large decision tree with 10 levels and 73 nodes, training MSE=0.00168, testing MSE=0.00371

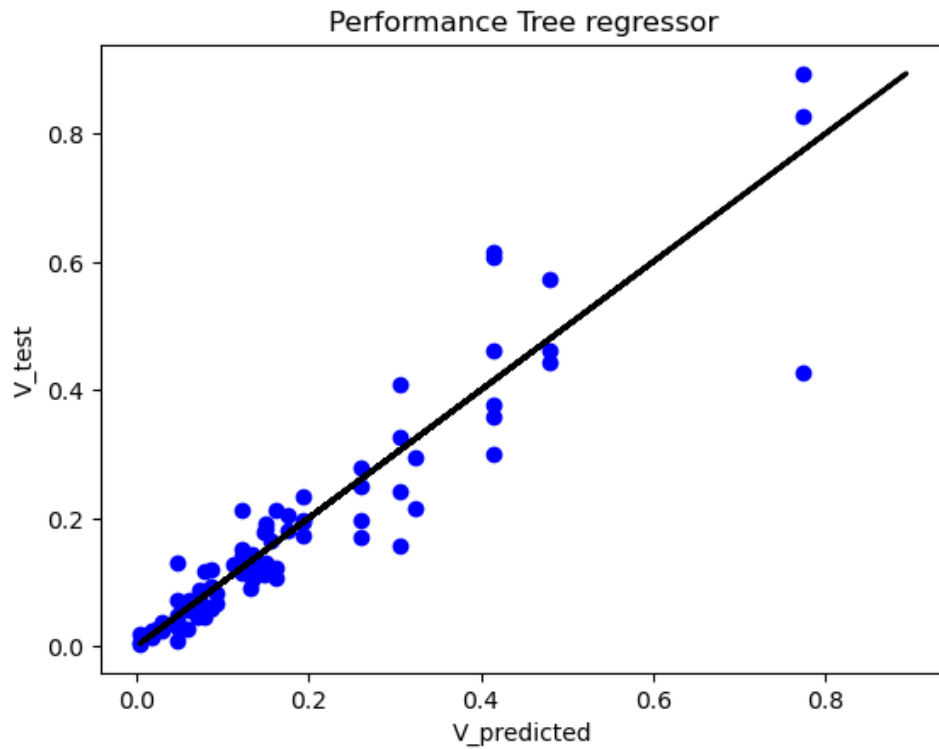


Figure 29: Prediction of large tree regressor model (MSE=0.00371)

Table 5: Hyperparameters and performance of different tree regressor models

Tree regressor	ccp_alpha	min_samples_leaf	min_samples_split	max_depth	MSE
Simple	-	5	-	2	0.00743
Large	-	5	-	100	0.00371
Best	0	5	15	8	0.00416

Figure 30 shows the best tree regressor plot. It is observed that the best tree initializes the splitting based on the feature `d` again. This further reinforces our hypothesis that `d` is the most important feature in the dataset. The model continues splitting based on `Fläche`, `rho_l` and `fcm_cyl` in that order, however it is curious that the model does not consider the profile shape features `Stütze_k`, `Stütze_q` and `Stütze_r` at all in order to split any nodes. This might mean that the profile shape of the test elements doesn't play a major role in pinching.

Figure 31 shows the prediction of the best decision tree regressor, with a comparable accuracy (MSE) to that of the previously mentioned linear and non-linear regression models. Even though these are good results, another class of decision trees can be modelled in order to get even better results. This class of decision trees are known as random forest which a combination of multiple decision trees.

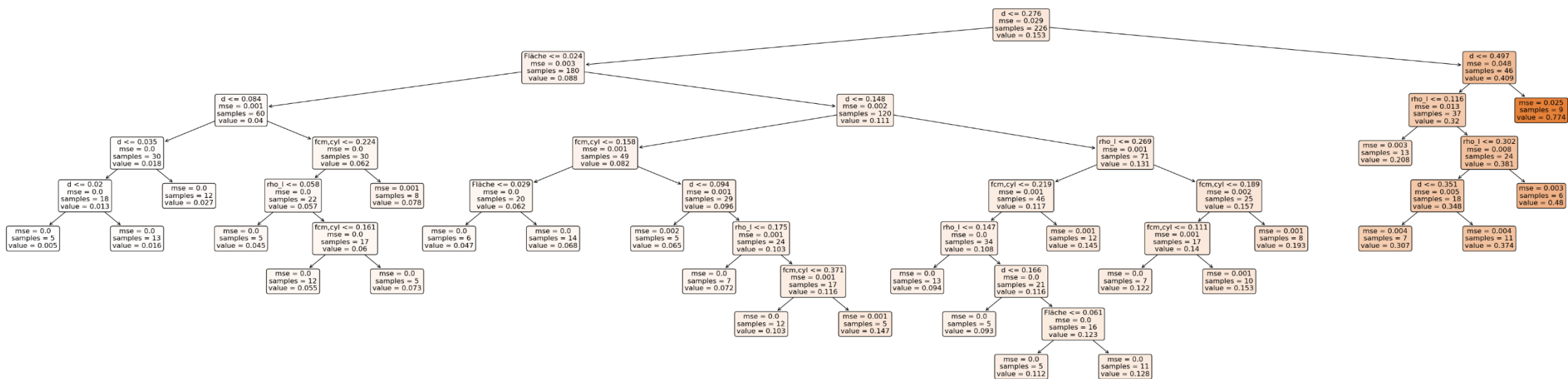


Figure 30: Best decision tree with 8 levels and 51 nodes, training MSE=0.0019, testing MSE=0.00416

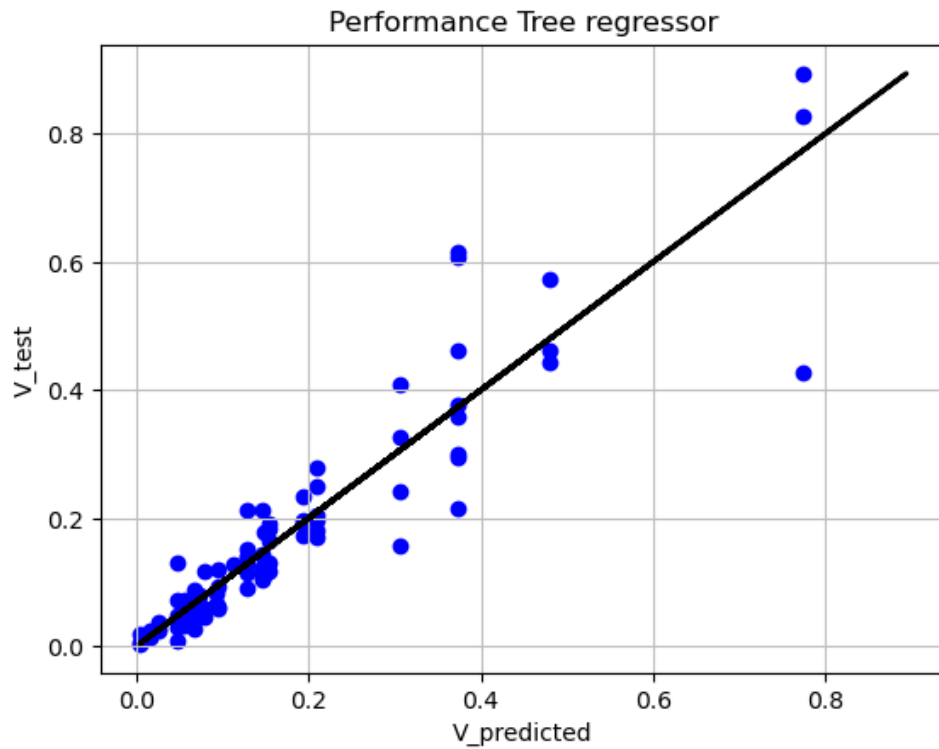


Figure 31: Prediction of best tree regressor model (MSE=0.00416)

In order to quantify relative importance of features according to the best decision tree regressor, a feature importance analysis is carried out. According to Figure 32, d , ρ_{ho_1} , Fläche and f_{cm_cyl} are the most important features and $Stütze_k$, $Stütze_q$ and $Stütze_r$ don't play a role in the splitting at all.

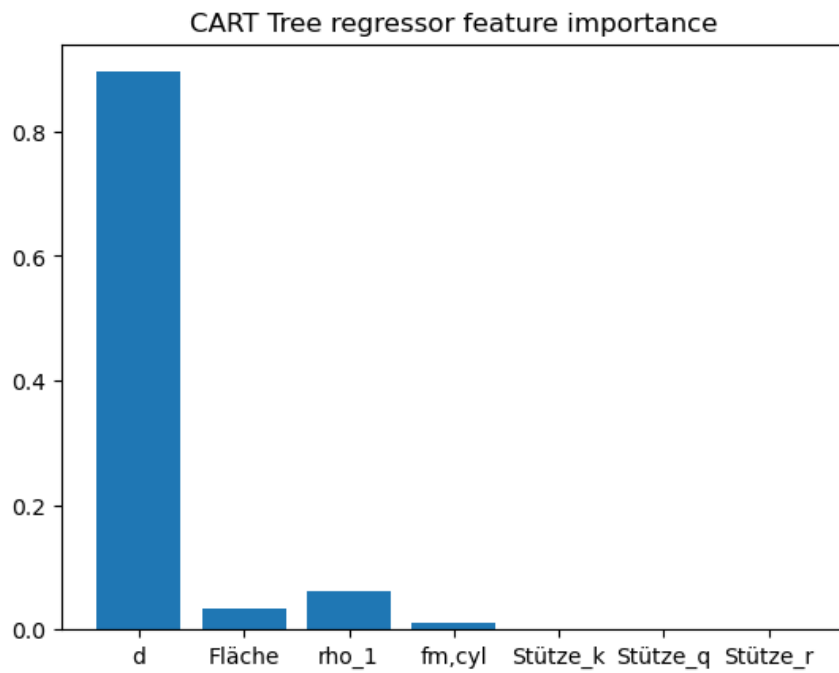


Figure 32: Feature importance according to best tree regressor

5.3 Random Forest Model

A random forest model is an ensemble of decision trees. Multiple decision trees are averaged in order to build a more robust model which has a better generalization performance and is less susceptible to overfitting.

As a first step, a generic random forest model with 1000 trees is created in order to make predictions. The resulting model has a much better performance than the best tree regressor model, with an improvement of 49.5%. In a second step, we tune the random forest hyperparameters to create the best random forest model. The best random forest model uses 200 trees and uses all 6 features for splitting. The predictive performance of both models can be seen in Figures 33 and 34 respectively.

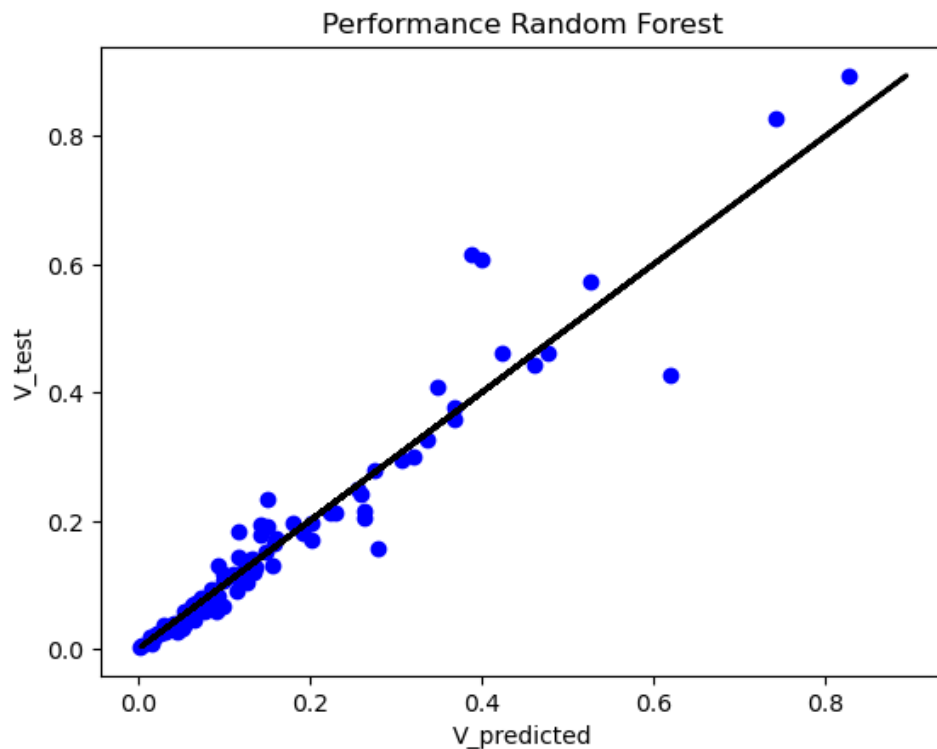


Figure 33: Prediction of first random forest model (MSE=0.0021, training MSE=0.00034)

A similar feature importance analysis is also carried out for the best random forest model. According to Figure 35, `d`, `Fläche`, `fcm_cyl` and `rho_l` are the most important features and `Stütze_k`, `Stütze_q` and `Stütze_r` don't play a role in the splitting at all. The result of the feature importance analysis is similar to the decision tree regressor feature importance, and we can rule out the importance and therefore inclusion of the element profile-shape in future experiments or code formulas.

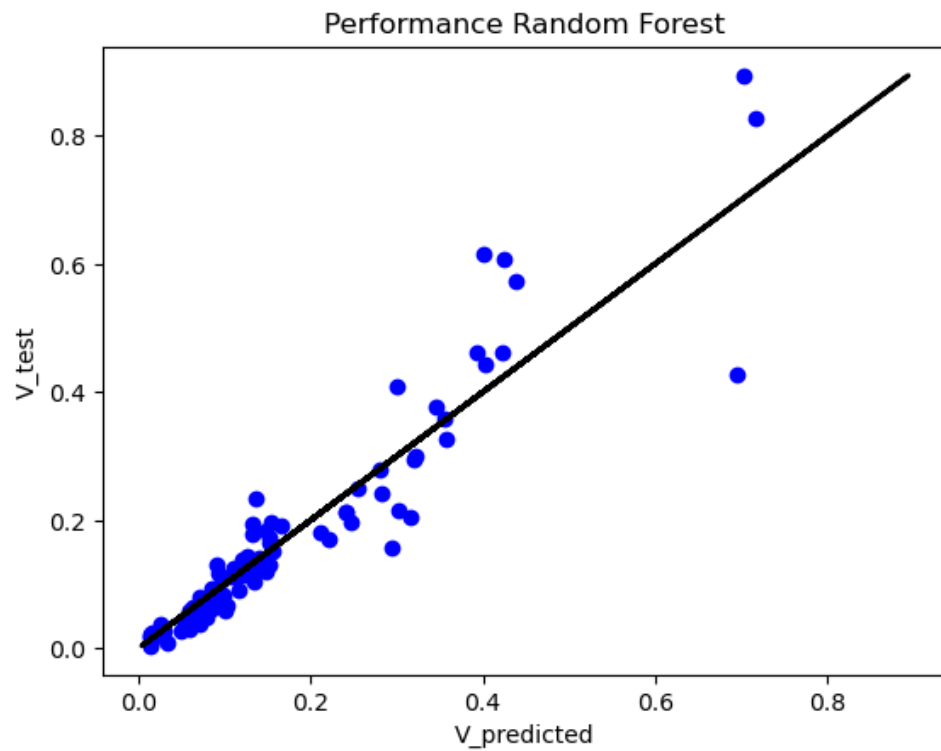


Figure 34: Prediction of best random forest model (MSE=0.0034, training MSE=0.00188)

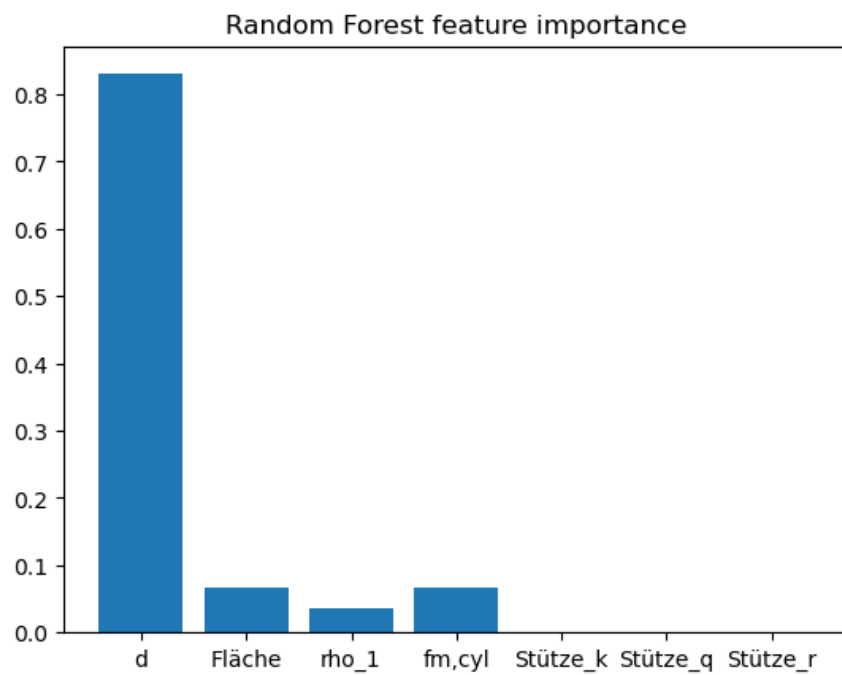


Figure 35: Feature importance according to best random forest model

5.4 Comparing Eurocode to Random Forest Model

To compare the two models the safety factor of 1.5 is removed from the Eurocode and $f_{ck} = f_{cm,cyl} - 8$ [N/mm^2] is used. On this basis the two models are compared. The blue points show the prediction of the model, and the green points show the Eurocode values. The relation between $V_{prediction}$ and V_{Rd_Code} shows a factor of 1.22 with a small negligible intersection offset. In conclusion we can say that using the developed non-linear regression model we can increase the punching resistance by 20 % with respect to the Eurocode.

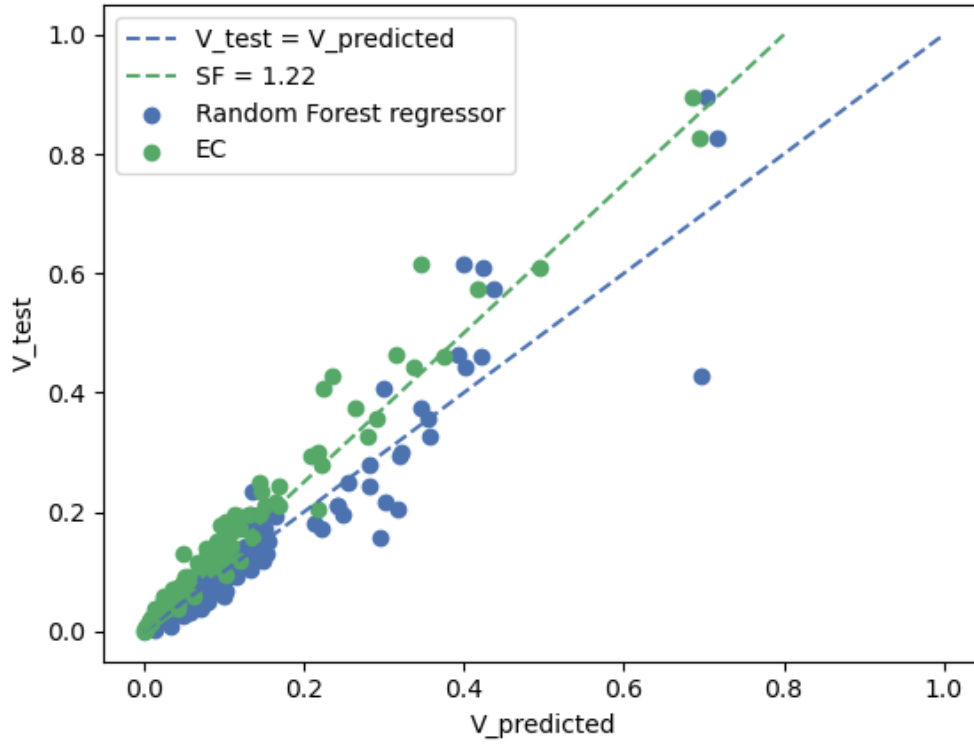


Figure 36: Random forest model predictions vs. Eurocode values

6 EuroCode

According to the Eurocode DIN EN 1992-1-1, the shear punching resistance of a slab is calculated as

$$v_{Rd,c} = C_{Rd,c} \cdot k \cdot \sqrt[3]{\rho_l \cdot 100 \cdot f_{ck}} + k_1 \cdot \sigma_{cp}$$

where:

$C_{Rd,c}$ = empirical factor,

$$k = 1 + \sqrt{\frac{200}{d}} \leq 2.0,$$

$$\rho_l \leq 0.02,$$

σ_{cp} = concrete stress state

Since we have no information about the concrete stress state (simply because we lack the data), the formula is reduced to:

$$v_{Rd,c} = C_{Rd,c} \cdot \left[\min \left(1 + \sqrt{\frac{200}{d}}, 2.0 \right) \right] \cdot \sqrt[3]{\min(\rho_l, 0.02) \cdot 100 \cdot f_{ck}}$$

The Eurocode proposes a $C_{Rd,c}$ of 0.12, which already includes a safety factor of 1.5. To get a better feeling for the formula and its capacity to accurately predict the punching resistance, the only empirical parameter $C_{Rd,c}$ was optimized to minimize the mean squared error (MSE). With this, the inherent structure of the formula is not altered, the formula is simply multiplied by an additional factor in order to get rid of unwanted safety in the prediction, because in the end we want to predict the punching strength as accurate as possible, without having any safety margin. As the plot in Figure shows, an optimized $C_{Rd,c}$ of 0.229 yields an accuracy measured with an MSE of 0.00885.

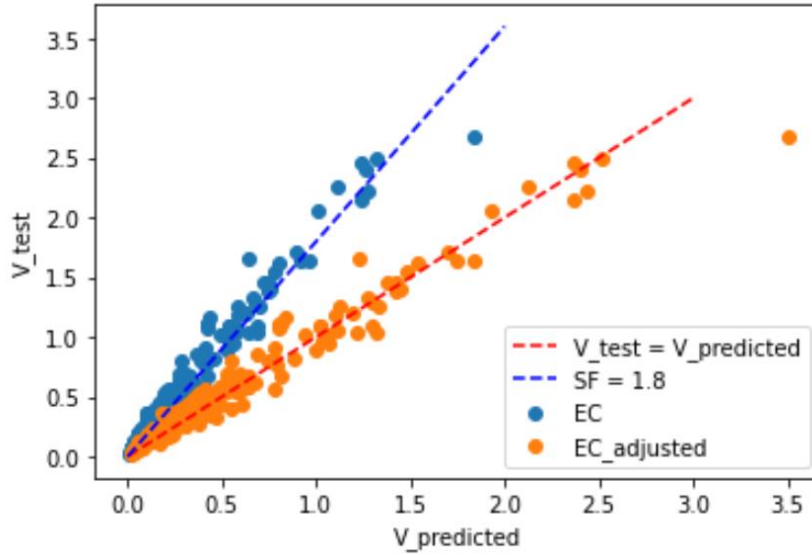


Figure 37: Eurocode performance for standard and adjusted formula

In order to compare the different regression models to the Eurocode formula, the Eurocode values need to be scaled in the same manner all other input values were scaled for the model training. If we scale the adjusted formula accordingly, we obtain the scatter plot presented in Figure . The measured accuracy yields an MSE of 0.0013. This is the target value to compare the regression models to.

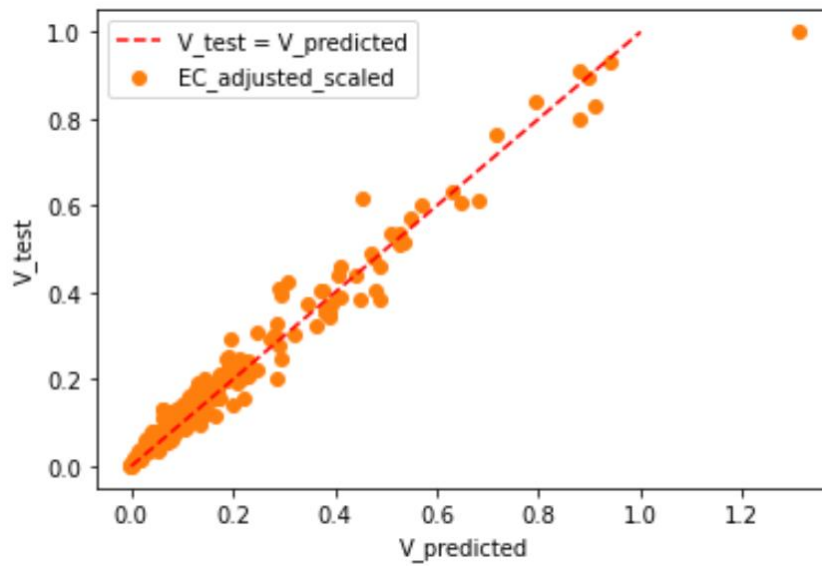


Figure 38: Eurocode performance in scaled dimensions with adjusted formula

6.1 Sensitivity Analysis of Eurocode

The aim of this part is not to analyze the Euro Code formula in depth, but more to have a general intuition of the impact on the different parameters. The same feature as in the ML models are chosen, as a basis assumption the mean value from the provided data is calculated for every feature (see Table 6). In every calculation step only one feature with respect to the reference values in Table is change +/- 20% and the percent of the resistance change is then plotted in Figure . One can observe a similar tendency in feature importance as in the ML Models.

Table 6: Mean Values of features

d	122 [mm]
Fläche	495.4 [cm ²]
rho	1.3 %
f _{cm, cyl}	36.1 [N/mm ²]

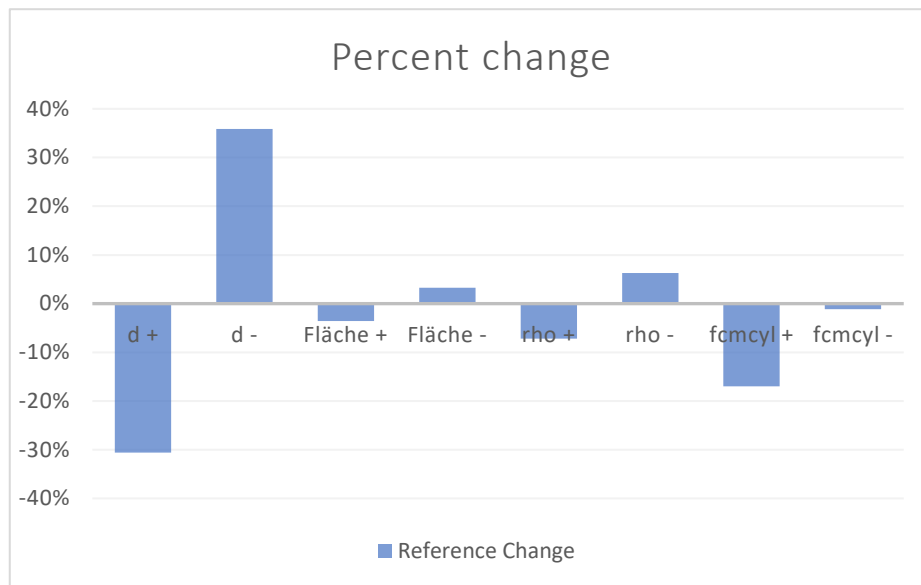


Figure 39: Feature sensitivity of Eurocode

7 Conclusion

In Conclusion one can see that the nonlinear regression models perform generally better than linear ones. Further the main performance indicator taken into consideration for this report was the MSE. In Table 5 a comparison of the investigated models is shown. The SVR is able to outperform Euro Code prediction in terms of MSE, but some loss of explain ability coming up compared to the Euro Code Model. Furthermore in Figure a comparison in terms of mean value and standard deviation is stated which was adopted from literature review [2]. For the comparison the mean value and standard deviation is normalized with μ_{Model}/μ_{Test} the same holds for the standard variation $\sigma_{model}/\sigma_{test}$. The Euro Code clearly has the most precise estimation with the lowest span of values, nevertheless, the estimation is slightly conservative and under the effective mean value. A reason for the low spread of the Euro Code values could be the limitation of the statical depth and the reinforcement amount for high values.

Table 5: Overview of model performance

	EC/V_test	SVR/V_test	NLR/V_test	RF/V_test
Mean	0.84	1.11	1.01	1.16
St. dev.	0.24	0.55	1.00	0.52
	EC	SVR	NLR	RF
MSE [%]	1.3	0.7	2.5	3.4

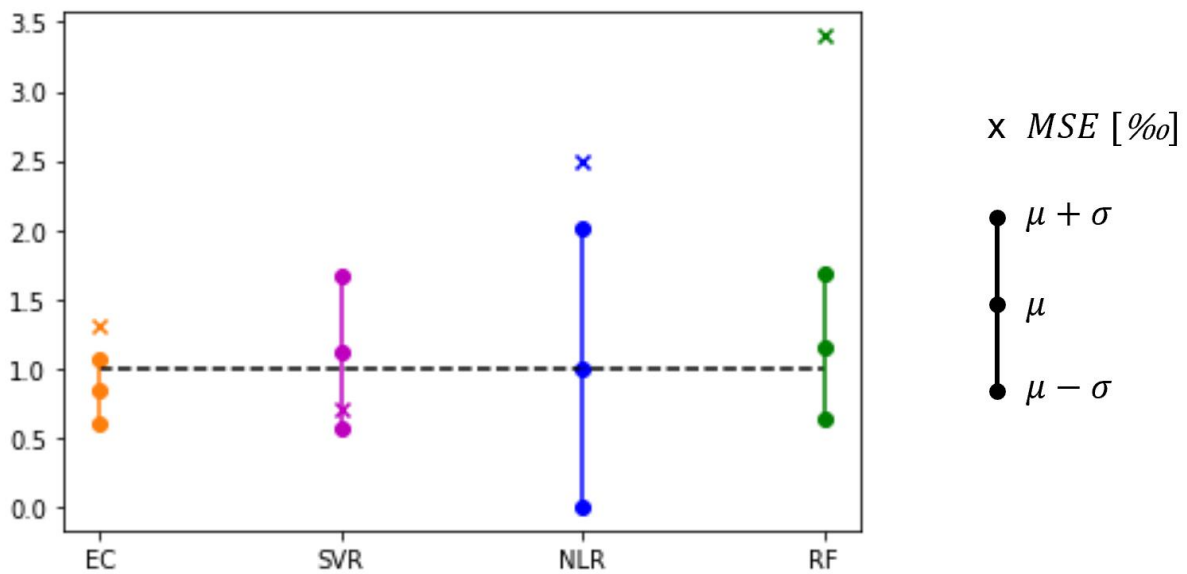


Figure 40: Model comparison

8 Outlook

In further steps a more detailed non-linear regression could be investigated, for example by not only optimize the coefficients but also investigate exponents to capture the non-linearity better but still remain in a explainable region. In addition, possible points for feature limitations can be searched using a decision tree and try to combine this with the non-linear model or the SVR. The result could then be an optimized model for the calculation of the load-bearing resistance. Nevertheless, an empirical model would have been generated. Another approach could be to combine mechanical properties with empirical test data.

9 References

- [1] M. Ricker, C. Siburg, Punching shear strength of flat slabs: Critical review of Eurocode 2 and fib Model Code 2010 design provisions, February 2016
- [2] H. D. Nguyen, G. T. Truong, M. Shin, Development of extreme gradient boosting model for prediction of punching shear resistance of r/c interior slabs, May 2021
- [3] T. Nguyen-Sy et. al., Predicting the compressive strength of concrete from its compositions and age using the extreme gradient boosting method, 2020
- [4] S. Mangalathu, H. Shin, E. Choi, J-S. Jeon, Explainable machine learning models for punching shear strength estimation of flat slabs without transverse reinforcement, July 2021
- [5] A. Elshafey, E. Rizk, H. Marzouk, M. Haddara, Prediction of punching shear strength of two-way slabs, May 2011

10 List of Figures

Figure 1 normal and adjusted EC formula	1
Figure 2 Heat-map showing available data (Yellow: missing data, Blue: available data).....	4
Figure 3 Colinearity Matrix	6
Figure 4 Histograms of normalized feature distribution	7
Figure 5 Box-plot of normalized features	7
Figure 6 Summary of linear regression model	8
Figure 7 Prediction of linear regression model (MSE=0.004092).....	9
Figure 8 Summary of non-linear regression model	10
Figure 9 Prediction of non-linear regression model (MSE=0.002473).....	11
Figure 10 Shapely Values of non-linear regression model.....	11
Figure 11 Prediction of ridge regression model (MSE=0.004579)	12
Figure 12 Prediction of elastic net regression model (MSE=0.005737).....	13
Figure 13 Prediction of lasso regression model (MSE=0.004540).....	13
Figure 14 Comparison of different linear and non-linear regression models.....	14
Figure 15 Non-linear regression model predictions vs. Eurocode values	15
Figure 16 Model performance for the linear kernel function	16
Figure 17 Scatterplot of input variables and support vectors for the linear kernel model	17
Figure 18 Scatterplots of input variables and support vectors for the polynomial kernel model	18
Figure 19 Model performance for the polynomial kernel function	18
Figure 20 Model performance for the radial basis kernel function	19
Figure 21 Scatterplots of input data and support vectors for the RBF kernel model	20
Figure 22 RBF model predictions vs. Eurocode values.....	22
Figure 23 Distribution of experiment counts among researchers in dataset	23
Figure 24 Distribution of experiment counts among filtered researchers in dataset	24
Figure 25 Distribution of experiment counts among filtered researchers in training subset	24
Figure 26 Distribution of experiment counts among filtered researchers in testing subset	25
Figure 27 Simple decision tree with 3 levels and 7 nodes, training MSE=0.00437, testing MSE=0.00743.....	26
Figure 28 Large decision tree with 10 levels and 73 nodes, training MSE=0.00168, testing MSE=0.00371.....	27
Figure 29 Prediction of large tree regressor model (MSE=0.00371).....	28
Figure 30 Best decision tree with 8 levels and 51 nodes, training MSE=0.0019, testing MSE=0.00416..	29
Figure 31 Prediction of best tree regressor model (MSE=0.00416).....	30

Figure 32 Feature importance according to best tree regressor	30
Figure 33 Prediction of first random forest model (MSE=0.0021, training MSE=0.00034).....	31
Figure 34 Prediction of best random forest model (MSE=0.0034, training MSE=0.00188).....	32
Figure 35 Feature importance according to best random forest model.....	32
Figure 36 Random forest model predictions vs. Eurocode values	33
Figure 37 Eurocode performance for standard and adjusted formula	34
Figure 38 Eurocode performance in scaled dimensions with adjusted formula	35
Figure 39 Feature sensitivity of Eurocode	36
Figure 40 Model comparison.....	37

11 List of Tables

Table 1 Features of Dataset	3
Table 2 Data Overview	5
Table 3 Variance Inflation Factor	6
Table 4 Optimized model hyperparameter and their effect on the model accuracy	21
Table 5 Hyperparameters and performance of different tree regressor models	28
Table 6 Mean Values of features.....	35